SITE E-COMMERCE AVEC SYMFONY

CEF

Mialy RABARISON

TABLE DES MATIÈRES

1 Présentation du projet **5**
1.1 Pages et fonctionnalités demandées 5
1.2 Ressources fournies 5
1.3 Plan de réalisation 6
2 Outils utilisés **6**
3 Préparation de l’environnement de développement **7**
3.1 Installation et vérification des outils de développement 7
3.1.1 Ouverture du terminal dans Visual Studio Code 7
3.1.2 Vérification de PHP 7
3.1.3 Vérification de Composer 8
3.1.4 Vérification de Symfony CLI 9
3.1.5 Vérification de Git 10
3.1.6 Activation de l'extension ZIP de PHP 10
3.1.7 Conclusion générale 11
4 Création du projet Symfony **12**
4.1 Création et vérification du projet Symfony 12
4.1.1 Création du projet Symfony STUBBORN 12
4.1.2 Vérification de la version du framework Symfony 14
4.1.3 Vérification de la structure du projet 15
4.1.4 Conclusion générale 16
4.2 LANCEMENT DU PROJET SYMFONY STUBBORN 16
4.2.1 Présentation du serveur Web local Symfony 16
4.2.2 Démarrage du serveur Symfony 17
4.2.3 Vérification dans le navigateur 18
4.2.4 Conclusion générale 19
5 Création de la page d'accueil du projet Stubborn **19**
5.1 Création de la première page d'accueil 19
5.1.1 Présentation du fonctionnement d'une page Symfony 20
5.1.2 Création du contrôleur HomeController 20
5.1.3 Ouverture et analyse du fichier HomeController.php 22
5.1.4 Modification de la route de la page d’accueil 23
5.1.5 Vérification de la page d’accueil dans le navigateur 24
5.2 Personnalisation de la page d’accueil STUBBORN 25
5.2.1 Ouverture du template Twig de la page d’accueil 26
5.2.2 Présentation du template commun base.html.twig 26
5.2.3 Modification du titre de la page 26
5.2.4 Personnalisation du fond avec CSS 27
5.3 Intégration du logo officiel dans l’en-tête 27
5.3.1 Importation du logo 28
5.3.2 Création de l’en-tête dans base.html.twig 28
5.4 Création du menu de navigation 29

TABLE DES MATIÈRES (suite)

5.4.1 Ajout du menu dans base.html.twig prévu pour un visiteur non conecté 29
5.4.2 Structure complète de l'en-tête 30
5.4.3 Préparation du menu dynamique 30
5.5 Organisation du contenu de la page d'accueil 30
5.5.1 Organisation du template de la page d'accueil 31
5.5.2 Organisation de la section des produits 31
5.5.3 Organisation de la section des informations de la société 31
5.6 Création des fiches produits sur la page d'accueil 31
5.6.1 Création du dossier images 32
5.6.2 Intégration des trois fiches produits 32
5.7 Finalisation de la page d'accueil pour un utilisateur non connecté 33
6 Gestion des utilisateurs et authentification 33
6.1 Configuration de la base de données 34
6.1.1 Démarrage des services avec XAMPP 34
6.1.2 Création de la base de données 34
6.1.3 Configuration de la connexion dans Symfony 35
6.1.4 Vérification de la connexion 36
6.1.5 Résultat de cette étape 36
6.2 Création de l'entité User 36
6.2.1 Génération de l'entité User avec Symfony 37
6.2.2 Ajout des propriétés demandées par le cahier des charges 38
6.2.3 Structure de l'entité User 39
6.2.4 Création de la migration Doctrine 39
6.2.5 Exécution de la migration 40
6.2.6 Vérification des tables dans phpMyAdmin 40
6.2.7 Résultat de cette étape 41
6.3 Création du formulaire d'inscription 41
6.4 Vérification de l'adresse e-mail et activation du compte 42
6.5 Connexion et déconnexion des utilisateurs 43
6.6 Adaptation de l'interface selon l'état de connexion 43
7 Création de la boutique et des fiches produits 44
7.1 Création du contrôleur et des routes /products et /product/{id} 44
7.2 Création des données des 10 sweatshirts : noms, prix, images, tailles et stocks 45
7.3 Création de la page Boutique avec les 10 produits, conformément au wireframe 46
7.4 Mise en place du filtre par fourchette de prix 47
7.5 Création de la fiche individuelle d'un sweatshirt 48
7.5.1 Création de la route de consultation d'un produit 48
7.5.2 Création du template de la fiche produit 49
7.5.3 Mise en forme de la fiche produit 49
7.5.4 Vérification de la fiche individuelle 49
8 Création du panier STUBBORN 50

TABLE DES MATIÈRES (suite)

8.1 Mise en place du contrôleur du panier 50
8.2 Création de la page d'affichage du panier 50
8.3 Mise en place de la suppression des articles 51
8.4 Mise en forme de la page Panier 51
8.5 Vérification du fonctionnement du panier 51
9 Installation et configuration de Stripe 52
9.1 Installation de la bibliothèque Stripe 52
9.2 Création du service Stripe 53
9.3 Configuration du service dans Symfony 53
9.4 Vérification de la configuration Symfony 53
10 Installation de Stripe CLI et création du Sandbox de test 54
10.1 Installation de Stripe CLI 54
10.2 Création de l'environnement Stripe Sandbox 54
10.3 Configuration de la clé API de test 55
10.4 Vérification de la communication avec Stripe 55
11 Intégration du paiement Stripe au panier STUBBORN 56
11.1 Intégration du bouton de paiement Stripe 56
11.2 Création de la session de paiement Stripe 56
11.3 Réalisation d'un paiement fictif 56
11.4 Vérification et confirmation du paiement 57
11.5 Vérification du panier après paiement 57
12 Réalisation du Back-Office – STUBBORN 58
12.1 Objectif du Back-Office 58
12.2 Sécurisation de l'accès administrateur 60
12.3 Ajout d'un sweatshirt 61
12.4 Modification d'un sweatshirt 62
12.5 Mise en avant des sweatshirts 63
12.6 Suppression d'un sweatshirt 64
12.7 Navigation entre le Back-Office et la Boutique 65
12.8 Vérifications techniques 65
12.9 Bilan 65
13 Réalisation des tests unitaires avec PHPUnit 66
13.1 Objectif des tests unitaires 66
13.2 Test du calcul du prix d'un produit 66
13.3 Test unitaire de l'ajout au panier 67
13.4 Test d'achat avec un panier vide 68
13.5 Test d'achat avec un produit valide 69
13.6 Exécution et résultats de l'ensemble des tests 71
13.7 Bilan des tests unitaires 71

RÉALISATION DU DEVOIR
SITE E-COMMERCE STUBBORN

1 Présentation du projet

Le projet consiste à développer une boutique en ligne pour la marque de sweat-shirts Stubborn, à l'aide de Symfony, PHP, MySQL/MariaDB et Twig. Le travail sera réalisé étape par étape, avec une explication des commandes, du code et du rôle des fichiers.

1.1 Pages et fonctionnalités demandées

Page	Route	Fonction
Accueil	/	Présentation de Stubborn et de trois produits mis en avant ; menu adapté à la connexion
Connexion	/login	Authentification des clients
Inscription	/register	Création de compte et confirmation par e-mail
Boutique	/products	Catalogue et filtre par fourchette de prix
Détail produit	/product/{id}	Choix de la taille et ajout au panier
Panier	/cart	Articles, suppression, total et paiement
Administration	/admin	Gestion des produits, réservée à l'administrateur

Autres exigences : intégrer Stripe en mode test, réaliser des tests unitaires du panier et du paiement, documenter l'application, fournir un dépôt GitHub et présenter une démonstration locale.

1.2 Ressources fournies

Un fichier ZIP rassemble les éléments graphiques nécessaires à la réalisation du site.

Ressource	Contenu
Logo	1 logo Stubborn
Produits	10 images de sweat-shirts
Wireframes	8 maquettes : accueil connecté et non connecté, connexion, inscription, boutique, produit, panier et administration

Les vues devront respecter les wireframes. Les images des produits et le logo seront intégrés au projet, par exemple dans public/images/.

Les trois produits mis en avant sont Blackbelt, Pokeball et BornInUsa. Chaque sweat-shirt existe en XS, S, M, L et XL, avec au moins deux exemplaires par taille.

1.3 Plan de réalisation

Étape	Travail prévu
1	Préparer et vérifier l'environnement de développement
2	Créer le projet Symfony Stubborn et vérifier son démarrage
3	Créer la base de données MySQL/MariaDB et ses entités
4	Construire les pages avec Twig, HTML et CSS selon les wireframes
5	Mettre en place l'inscription, la connexion et les rôles
6	Développer la boutique, les filtres et le panier
7	Intégrer Stripe en mode test et créer le back-office
8	Réaliser les tests, publier sur GitHub et finaliser la documentation

2 Outils utilisés

Outil	Rôle
Visual Studio Code	Éditeur de code et terminal intégré
PHP 8.2	Langage côté serveur
Symfony	Framework de développement web
Composer	Gestion des dépendances PHP
Symfony CLI	Création et exécution des projets Symfony
XAMPP	Environnement local et serveur MySQL/MariaDB
Git	Versionnement du code
GitHub	Hébergement et partage du dépôt

Pourquoi Visual Studio Code ?

Visual Studio Code permet de consulter l'arborescence du projet, d'écrire les fichiers PHP, Twig, HTML et CSS, d'utiliser un terminal intégré et d'installer des extensions adaptées à Symfony.

3 Préparation de l'environnement de développement

Objectif
Disposer d'un environnement local permettant de développer et d'exécuter l'application Symfony sur Windows.

3.1 Installation et vérification des outils de développement

3.1.1 Ouverture du terminal dans Visual Studio Code

Les opérations suivantes ont été réalisées :

1. Ouverture de Visual Studio Code.
2. Ouverture du dossier de travail Documents\Formation CEF\SYMFONY E-COMMERCE.
3. Sélection du menu **Terminal → Nouveau terminal**.
4. Vérification du répertoire courant dans PowerShell.

Les commandes de vérification ont ensuite été exécutées individuellement.

3.1.2 Vérification de PHP

Présentation

PHP est un langage de programmation exécuté côté serveur. Il permet de traiter les données et de développer les fonctionnalités dynamiques d'une application Web. Symfony est un framework développé en PHP.

Commande utilisée

php -v

Explication de la commande

- php : appelle l'interpréteur PHP.
- -v : affiche la version de PHP installée.

Résultat obtenu

- Version PHP : **8.2.12**
- OPcache : disponible.

La version PHP utilisée est compatible avec Symfony 7.4, qui nécessite PHP 8.2 ou une version supérieure.

Conclusion

PHP est correctement installé et accessible depuis le terminal de Visual Studio Code.

```
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE> php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.12, Copyright (c), by Zend Technologies
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE>
```

3.1.3 Vérification de Composer

Présentation

Composer est un gestionnaire de dépendances PHP. Il permet d'installer, de mettre à jour et de gérer les bibliothèques nécessaires au fonctionnement d'une application Symfony.

Commande utilisée

composer --version

Explication de la commande

- composer : appelle le gestionnaire de dépendances.
- --version : affiche la version installée.

Résultat obtenu

- Composer : **version 2.10.3**
- PHP utilisé par Composer : **8.2.12**
- Chemin de PHP : C:\xampp\php\php.exe

```
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE> composer --version
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE>
```

Conclusion

Composer est correctement installé et accessible depuis Visual Studio Code. Il utilise la version de PHP configurée dans XAMPP.

3.1.4 Vérification de Symfony CLI

Présentation

Symfony CLI (Command Line Interface) est un outil en ligne de commande permettant de faciliter le développement des applications Symfony. Il permet notamment de créer des projets Symfony, de démarrer un serveur local et d'exécuter certaines commandes utiles au développement.

Il permet notamment :

- De créer un nouveau projet Symfony.
- De démarrer un serveur Web local.
- De vérifier certains prérequis techniques.
- De faciliter l'exécution de commandes liées au projet.

Commande utilisée

symfony -v

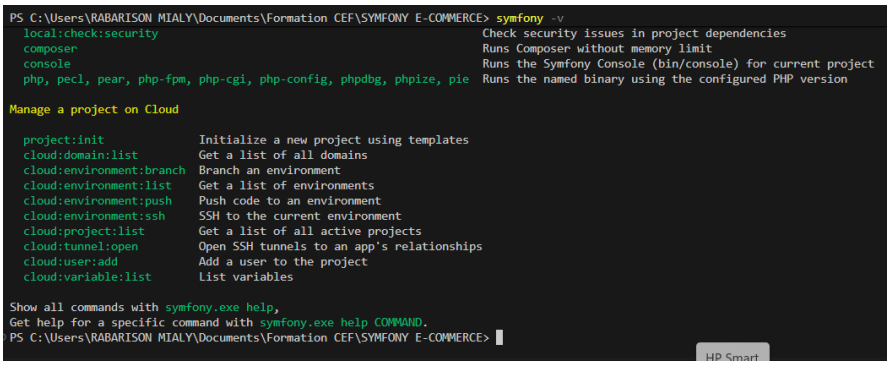
Explication de la commande

- symfony : appelle Symfony CLI.
- -v : demande l'affichage des informations de version.

Résultat obtenu

Symfony CLI : **version 5.22.0**.

Cette version correspond à l'outil Symfony CLI et non à la version du framework Symfony utilisée dans le projet.



Conclusion

Symfony CLI est installé et accessible depuis le terminal. Il pourra être utilisé pour créer le projet STUBBORN et lancer son serveur Web local.

3.1.5 Vérification de Git

Présentation

Git est un système de gestion de versions permettant de suivre et d'enregistrer les modifications apportées aux fichiers d'un projet.

Il permet notamment de conserver l'historique des modifications, de revenir à une version précédente et de préparer le partage du code source.

Différence entre Git et GitHub

- **Git** : outil de gestion des versions installé localement.
- **GitHub** : plateforme en ligne permettant d'héberger et de partager des dépôts Git.

Commande utilisée

git --version

Explication de la commande

- git : appelle le programme Git.
- --version : affiche sa version.

Résultat obtenu

Git : **version 2.46.0**.

```
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE> git -v
git version 2.46.0.windows.1
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE>
```

Conclusion

Git est correctement installé et accessible depuis Visual Studio Code. Il pourra être utilisé pour gérer les versions du projet STUBBORN et publier le code source sur GitHub.

3.1.6 Activation de l'extension ZIP de PHP

Objectif

Vérifier et activer l'extension ZIP de PHP afin de faciliter l'installation des dépendances du projet Symfony avec Composer.

Explication théorique

L'extension ZIP permet à PHP de manipuler des archives compressées au format ZIP.

Composer peut utiliser cette extension pour extraire les bibliothèques téléchargées lors de l'installation des dépendances.

Les extensions PHP sont configurées dans le fichier php.ini.

Dans ce fichier, le point-virgule ; placé au début d'une ligne indique que la directive est commentée et n'est pas appliquée.

Réalisation pratique

1. Ouverture du fichier C:\xampp\php\php.ini.
2. Recherche de la directive relative à l'extension ZIP.
3. Suppression du point-virgule si l'extension était désactivée.
4. Enregistrement du fichier.

Avant modification :

;extension=zip

Après modification :

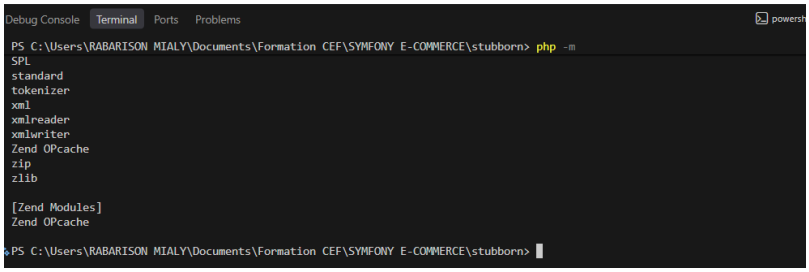
extension=zip

Vérification

La commande suivante permet de vérifier que l'extension ZIP est chargée par PHP :

php -m

Elle affiche la liste des extensions PHP actives. L'extension zip doit apparaître dans cette liste après activation et ouverture d'un nouveau terminal.



Conclusion

L'activation de l'extension ZIP permet à PHP de prendre en charge les archives ZIP et facilite l'installation des dépendances avec Composer.

3.1.7 Conclusion générale

Les principaux outils nécessaires au développement du site e-commerce STUBBORN ont été installés et vérifiés dans l'environnement Windows 10 et Visual Studio Code.

PHP assurera l'exécution du code de l'application, Composer gérera les dépendances, Symfony CLI facilitera le développement et l'exécution locale du projet, tandis que Git permettra le suivi des modifications du code source.

Ces vérifications constituent une étape préalable à la création et à la configuration du projet Symfony STUBBORN.

4 Création du projet Symfony

4.1 Création et vérification du projet Symfony

Objectif

L'objectif de cette étape est de créer une application Symfony nommée STUBBORN, qui servira de base au développement du site e-commerce de vente de sweatshirts.

Cette étape consiste à créer le projet, à vérifier la version du framework Symfony installé et à examiner la structure des dossiers et fichiers générés automatiquement.

Présentation

Symfony est un framework PHP permettant de développer des applications Web structurées.

Il utilise notamment :

- **Les contrôleurs** : traitent les requêtes et déterminent les actions à exécuter.
- **Twig** : moteur de templates permettant de générer les pages HTML.
- **Doctrine ORM** : facilite la communication entre l'application et la base de données.
- **Security** : fournit les fonctionnalités d'authentification et de gestion des autorisations.
- **Routing** : permet d'associer une URL à une action d'un contrôleur.

4.1.1 Création du projet Symfony STUBBORN

Réalisation pratique

Les opérations suivantes ont été réalisées :

1. Ouverture de Visual Studio Code.
2. Ouverture du dossier de travail Documents\Formation CEF\SYMFONY E-COMMERCE.
3. Ouverture du terminal intégré de VS Code.
4. Exécution de la commande de création du projet.

Commande utilisée

symfony new stubborn --version="7.4.*" --webapp

Explication de la commande

- symfony new : crée un nouveau projet Symfony.
- stubborn : définit le nom du dossier du projet.
- --version="7.4.*" : sélectionne la version 7.4 du framework Symfony.
- --webapp : installe les dépendances couramment nécessaires au développement d'une application Web.

Résultat obtenu

Le terminal a affiché le message :

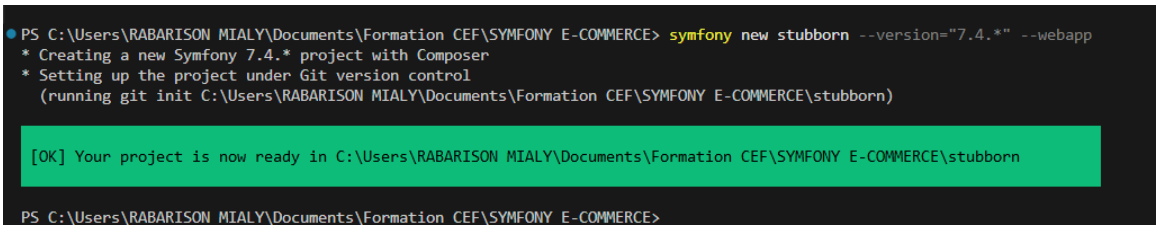
[OK] Your project is now ready

Le projet a été créé dans le répertoire :

Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn

Un dépôt Git local a également été initialisé automatiquement.

Le projet n'a pas besoin d'être placé dans C:\xampp\htdocs, car Symfony CLI dispose de son propre serveur Web local.



Structure

```
SYMPHONY E-COMMERCE/
  stubborn/
    config/
    public/
    src/
    templates/
    vendor/
    composer.json
```

Conclusion

Le projet Symfony STUBBORN a été créé avec succès. Les fichiers et dossiers nécessaires au développement de l'application ont été générés automatiquement.

4.1.2 Vérification de la version du framework Symfony

Objectif

Vérifier que le framework Symfony est correctement installé dans le projet STUBBORN et que sa version correspond à celle sélectionnée lors de sa création.

Commande utilisée

php bin/console --version

Cette commande est exécutée depuis le dossier stubborn.

Explication de la commande

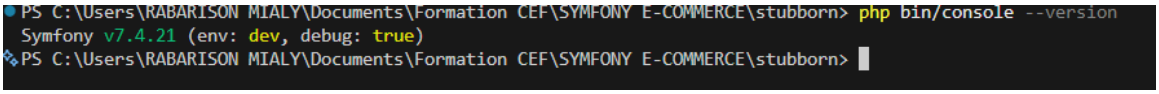
- php : appelle l'interpréteur PHP.
- bin/console : permet d'exécuter les commandes internes de Symfony.
- --version : affiche la version du framework Symfony installé.

Résultat attendu

Le terminal doit afficher une version de Symfony appartenant à la branche 7.4.x.

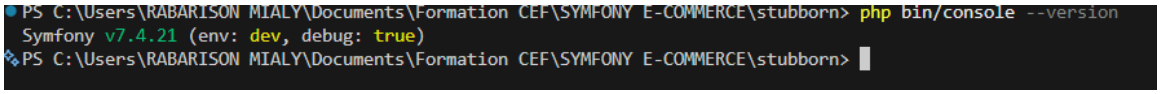
Il est important de distinguer la version du framework Symfony de celle de Symfony CLI :

- Symfony CLI 5.22.0 : outil de gestion et de développement.
- Symfony 7.4.x : framework utilisé pour développer le projet STUBBORN.



Conclusion

Cette vérification permet de confirmer la version du framework utilisée dans l'application.



4.1.3 Vérification de la structure du projet

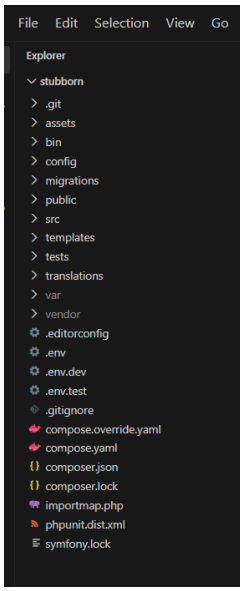
Objectif

Vérifier la présence des principaux dossiers et fichiers créés automatiquement par Symfony et comprendre leur rôle dans l'organisation du projet STUBBORN.

Réalisation pratique

Après la création du projet, le dossier stubborn a été ouvert dans l'Explorateur de fichiers de Visual Studio Code.

Les principaux dossiers suivants ont été identifiés :



Dossier	Rôle dans notre projet Stubborn
`.git`	Enregistre l'historique des modifications du projet.
`assets`	Contiendra notamment les fichiers CSS et JavaScript.
`bin`	Contient la console Symfony.
`config`	Contient la configuration de l'application.
`migrations`	Contiendra les modifications de structure de la base de données.
`public`	Contiendra notamment les images publiques et le point d'entrée du site.
`src`	Contiendra notre code PHP : contrôleurs, entités, services, etc.
`templates`	Contiendra les pages Twig : accueil, produits, panier, connexion...
`tests`	Contiendra les tests automatisés.
`translations`	Contiendra les fichiers de traduction.
`var`	Contiendra le cache et les journaux techniques.
`vendor`	Contient les bibliothèques installées par Composer.

Principaux fichiers de configuration

- .env : contient les variables d'environnement générales.
- composer.json : définit les dépendances PHP du projet.
- composer.lock : conserve les versions exactes des dépendances installées.
- symfony.lock : contient les informations relatives aux recettes Symfony Flex.
- importmap.php : configure les modules JavaScript utilisés avec Importmap.
- public/index.php : point d'entrée de l'application Web.

Résultat obtenu

L'exploration du projet dans Visual Studio Code a confirmé la présence des principaux dossiers et fichiers nécessaires au fonctionnement d'une application Symfony.

Conclusion

La structure initiale du projet STUBBORN est correctement créée. Elle permettra d'organiser le développement de l'application en séparant le code PHP, les vues Twig, les ressources publiques, les configurations et les fichiers liés à la base de données.

4.1.4 Conclusion générale

La création du projet Symfony STUBBORN a été réalisée avec succès à l'aide de Symfony CLI.

La structure initiale de l'application est en place et les principaux dossiers et fichiers ont été identifiés.

Cette étape a constitué la base technique du développement du site e-commerce.

Le serveur Web local a ensuite été lancé afin de vérifier l'affichage de l'application dans un navigateur.

4.2 LANCEMENT DU PROJET SYMFONY STUBBORN

Objectif

L'objectif de cette étape est de démarrer le serveur Web local de Symfony afin de vérifier que le projet STUBBORN fonctionne correctement.

Le lancement du serveur permettra d'accéder à l'application depuis un navigateur Web et de contrôler son fonctionnement avant de commencer le développement des fonctionnalités du site e-commerce.

4.2.1 Présentation du serveur Web local Symfony

Symfony CLI dispose d'un serveur Web local appelé **Symfony Local Web Server**, destiné au développement des applications Symfony.

Ce serveur permet d'exécuter une application Symfony directement sur l'ordinateur, sans avoir besoin de la publier sur Internet.

Il est important de distinguer :

- **Symfony CLI** : outil en ligne de commande permettant de créer et de gérer les projets Symfony et de démarrer un serveur local.
- **Symfony Local Web Server** : serveur de développement fourni par Symfony CLI permettant d'exécuter et de tester l'application dans un navigateur.

Contrairement à Apache fourni avec XAMPP, Symfony Local Web Server permet de développer l'application sans avoir besoin de démarrer Apache.

Une fois le serveur lancé, l'application est accessible depuis l' adresse locale :

http://127.0.0.1:8000

Cette adresse correspond à l'ordinateur local et permet de tester l'application pendant son développement.

4.2.2 Démarrage du serveur Symfony

Réalisation pratique

Les opérations suivantes ont été réalisées :

1. Ouverture du projet stubborn dans Visual Studio Code.
2. Ouverture du terminal intégré de VS Code.
3. Vérification que le terminal est positionné dans le dossier stubborn.
4. Exécution de la commande de démarrage du serveur.

Commande utilisée

symfony server:start

Explication de la commande

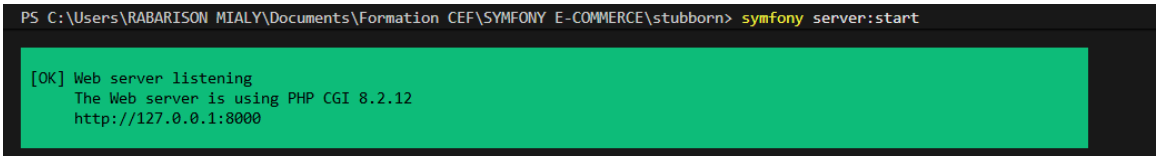
- symfony : appelle l'outil Symfony CLI.
- server:start : démarre le serveur Web local de développement.

Résultat obtenu

Le terminal a affiché un message confirmant le démarrage du serveur, ainsi que l'adresse locale permettant d'accéder à l'application :

http://127.0.0.1:8000

Figure 4.1 — Démarrage du serveur Web local Symfony dans le terminal de Visual Studio Code.



Explication des messages d'avertissement

Certains avertissements peuvent apparaître lors du démarrage du serveur :

- **TLS** : indique que le certificat local permettant d'utiliser HTTPS n'est pas installé. Le développement peut néanmoins se poursuivre en HTTP.
- **Production** : rappelle que le serveur Symfony CLI est destiné au développement et non à l'hébergement d'un site public.
- **127.0.0.1** : correspond à l'adresse locale de l'ordinateur. Le serveur est accessible uniquement depuis la machine utilisée, dans sa configuration par défaut.

Ces messages n'empêchent pas le fonctionnement du serveur dans notre environnement de développement.

Conclusion

Le serveur Web local du projet STUBBORN a été démarré avec succès. L'application peut désormais être testée depuis un navigateur.

4.2.3 Vérification dans le navigateur

Objectif

Vérifier que l'application Symfony répond correctement aux requêtes HTTP et qu'elle est accessible depuis le navigateur.

Réalisation pratique

Après le démarrage du serveur, l'adresse suivante a été saisie dans le navigateur :

http://127.0.0.1:8000

La page d'accueil provisoire de Symfony s'est affichée.

Résultat obtenu

Symfony affiche notamment le message suivant :

You are seeing this page because the homepage URL is not configured and debug mode is enabled.

Explication du message

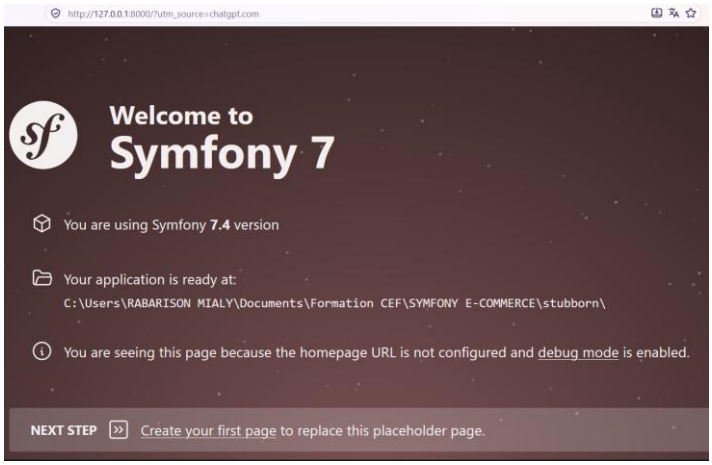
Ce message indique que :

- **Symfony fonctionne** : l'application répond correctement dans le navigateur.
- **La page d'accueil n'est pas encore configurée** : aucune route personnalisée n'a encore été définie pour l'URL /.
- **Le mode debug est activé** : l'application fonctionne en mode développement, ce qui facilite l'identification des erreurs.

Cette page provisoire apparaît automatiquement lorsque la route d'accueil n'a pas encore été configurée.

Elle confirme que le serveur Web local et l'application Symfony fonctionnent correctement.

Figure 4.2 — Affichage de la page provisoire de Symfony dans le navigateur.



Conclusion

L'application STUBBORN est accessible depuis le navigateur. Le serveur Web local fonctionne et permet d'exécuter le projet Symfony.

4.2.4 Conclusion générale

Le lancement du projet Symfony STUBBORN a été réalisé avec succès à l'aide du serveur Web local fourni par Symfony CLI.

L'application est accessible depuis le navigateur à l'adresse locale indiquée par le serveur.

La vérification de la page provisoire de Symfony confirme le bon fonctionnement de l'environnement de développement.

Le projet est désormais prêt pour la création de ses premières fonctionnalités, notamment les contrôleurs, les routes et les vues Twig nécessaires à la réalisation du site e-commerce STUBBORN.

5 Création de la page d'accueil du projet Stubborn

5.1 Création de la première page d'accueil

Objectif

L'objectif de cette étape est de créer la première page d'accueil du site e-commerce STUBBORN afin de remplacer la page provisoire affichée par Symfony.

Cette étape permet de découvrir **le fonctionnement des contrôleurs Symfony, du système de routage et du moteur de templates Twig.**

La page d'accueil constituera **le point d'entrée de la boutique en ligne.** Elle sera progressivement enrichie avec les éléments graphiques et les fonctionnalités définis dans le cahier des charges.

5.1.1 Présentation du fonctionnement d'une page Symfony

Dans Symfony, l'affichage d'une page Web repose principalement sur trois éléments :

- **La route (Route)** : associe une adresse URL à une méthode d'un contrôleur.
- **Le contrôleur (Controller)** : reçoit la requête HTTP et détermine la réponse à retourner.
- **Le template Twig** : génère le contenu HTML qui sera affiché dans le navigateur.

Lorsqu'un utilisateur accède à une adresse du site, Symfony identifie la route correspondante, exécute le contrôleur associé et peut utiliser un template Twig pour produire la page HTML.

Cette organisation sera utilisée pour créer la page d'accueil de STUBBORN.

5.1.2 Création du contrôleur HomeController

Présentation

Un contrôleur est une classe PHP chargée de traiter les requêtes et de déterminer la réponse envoyée au navigateur.

Dans notre projet, le contrôleur HomeController sera chargé de gérer la page d'accueil de la boutique STUBBORN.

Réalisation pratique

Les opérations suivantes ont été réalisées :

1. Conserver le premier terminal dans lequel le serveur Symfony fonctionne.
2. Dans Visual Studio Code, sélectionner Terminal → Nouveau terminal.
3. Vérifier que le terminal se trouve dans le dossier stubborn.
4. Exécuter la **commande de création du contrôleur.**

Commande utilisée

php bin/console make:controller HomeController

Explication de la commande

- php : appelle l'interpréteur PHP.
- bin/console : permet d'exécuter les commandes internes de Symfony.
- make:controller : génère automatiquement un contrôleur Symfony.
- HomeController : définit le nom du contrôleur à créer.

Résultat obtenu

Symfony a généré automatiquement deux fichiers :

- src/Controller/HomeController.php : contient le contrôleur de la page d'accueil.
- templates/home/index.html.twig : contient le template Twig associé au contrôleur.

Lors de la génération, Symfony a également proposé de créer un fichier de test pour le contrôleur.

La réponse no a été sélectionnée afin de se concentrer dans un premier temps sur la création et la compréhension de la page d'accueil.

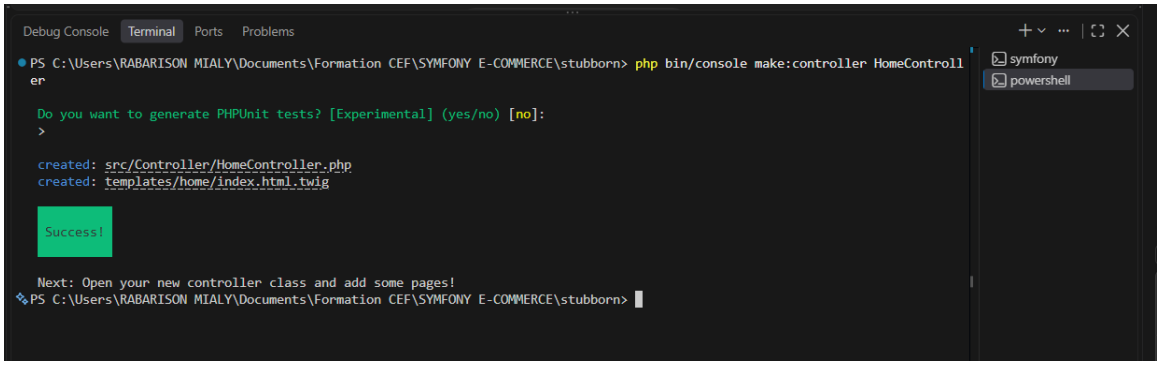


Figure 5.1 — Création du contrôleur HomeController : commande exécutée et message « Success! ».

Dans l'Explorateur de Visual Studio Code, le fichier HomeController.php apparaît dans le dossier **src/Controller.**

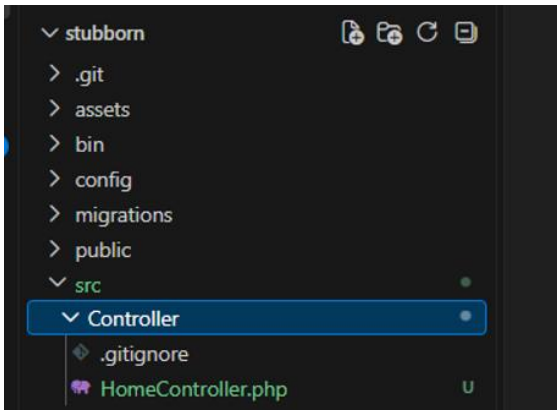


Figure 5.2 — Présence de HomeController.php dans le dossier src/Controller.

Conclusion

Le contrôleur HomeController et son template Twig ont été créés. Ils constituent les premiers éléments nécessaires à l'affichage de la page d'accueil du site STUBBORN.

5.1.3 Ouverture et analyse du fichier HomeController.php

Objectif

Ouvrir le contrôleur créé par Symfony et comprendre le code généré avant de le modifier.

Réalisation pratique

1. Dans l'Explorateur VS Code, ouverture du src.
2. Ouverture du sous-dossier Controller.
3. Clic sur HomeController.php.
4. Observation du contenu du fichier, notamment la route et la méthode index().

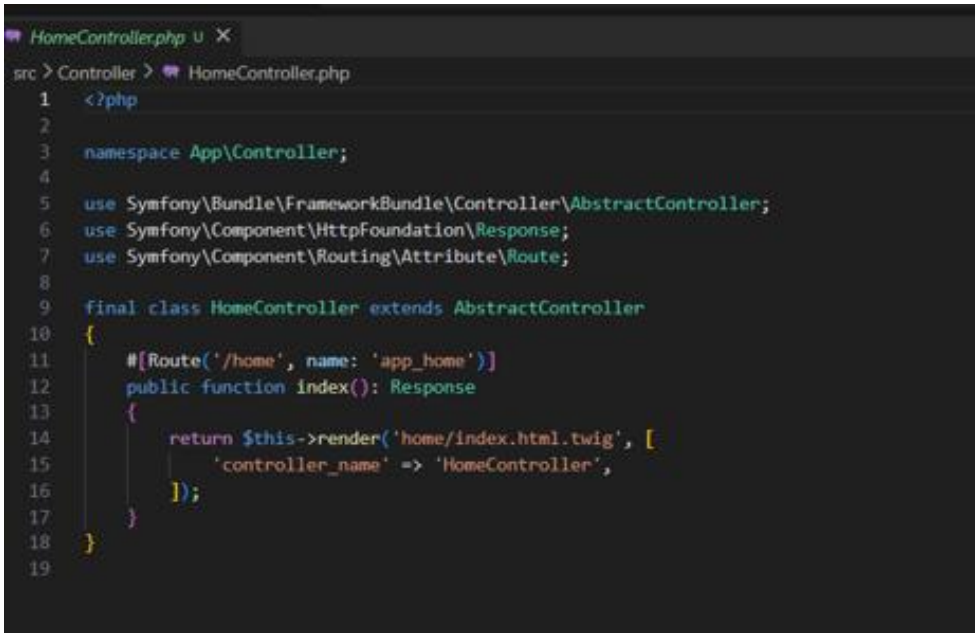


Figure 5.3 — Contenu initial de HomeController.php : route /home et appel au template Twig.

Explication du code observé

Élément	Explication
`namespace App\Controller;`	Indique que la classe appartient à l'espace de noms des contrôleurs.
`AbstractController`	Fournit des méthodes utiles aux contrôleurs Symfony, notamment `render()`.
`Response`	Représente la réponse HTTP envoyée au navigateur.
`Route`	Permet d'associer une URL à une méthode du contrôleur.
`HomeController`	Classe PHP qui gère notre page.
`index()`	Méthode exécutée lorsque la route correspondante est appelée.
`render()`	Génère une réponse HTML à partir d'un template Twig.

Analyse de la route initiale

```
#[Route('/home', name: 'app_home')]
```

Cette ligne signifie que Symfony associe cette page à l'adresse **http://127.0.0.1:8000/home.** Pour notre boutique STUBBORN, nous souhaitons cependant que la page d'accueil s'affiche directement à l'adresse http://127.0.0.1:8000/. Nous allons donc modifier la route.

Conclusion

L'ouverture du contrôleur a permis d'identifier la route initiale /home et de comprendre pourquoi elle doit être adaptée.

5.1.4 Modification de la route de la page d'accueil

Objectif

Configurer la route principale du site afin que la page d'accueil soit accessible directement à la racine de l'application.

Présentation

Dans Symfony, une route permet d'associer une URL à une méthode d'un contrôleur.

La route est définie dans notre contrôleur à l'aide de l'attribut PHP #[Route(...)].

Réalisation pratique

Le fichier suivant a été ouvert dans Visual Studio Code :

src/Controller/HomeController.php

- La route générée initialement était : #[Route('/home', name: 'app_home')]
- Cette route permettait d'accéder à la page depuis l'adresse : http://127.0.0.1:8000/home

Afin de définir cette page comme page d'accueil principale, la route a été modifiée comme suit :

#[Route('/', name: 'app_home')]

Explication du code

- #[Route(...)] : définit une route Symfony.
- '/' : désigne la racine du site, c'est-à-dire la page d'accueil.
- name: 'app_home' : attribue un nom à la route pour pouvoir la référencer dans l'application.

Résultat obtenu

La page d'accueil est désormais associée à l'adresse :

http://127.0.0.1:8000/

HomeController.php U | index.html.twig U | base.html.twig | app.css M

src > Controller > HomeController.php

```php
<?php

namespace App\Controller; //Indique que la classe appartient à l'espace de noms des contrôleurs.


use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;// AbstractController :Fournit des méthodes utiles aux contr
use Symfony\Component\HttpFoundation\Response; //Représente la réponse HTTP envoyée au navigateur
use Symfony\Component\Routing\Attribute\Route; //Permet d'associer une URL à une méthode du contrôleur.

final class HomeController extends AbstractController
{
    #[Route('/', name: 'app_home')] //Avant on a #[Route('/home', name: 'app_home')]
    // mais je veux mettre dans #[Route('/', name: 'app_home')]
    public function index(): Response //INDEX:Méthode exécutée lorsque la route correspondante est appelée.
    {
        return $this->render('home/index.html.twig', [ //render Méthode exécutée lorsque la route correspondante est appelée.
            'controller_name' => 'HomeController', //Classe PHP qui gère notre page.
        ]);
    }
}

```

Conclusion

La route principale du projet STUBBORN a été configurée. Elle permet d'accéder directement au contrôleur de la page d'accueil.

5.1.5 Vérification de la page d'accueil dans le navigateur

Objectif

Vérifier que la nouvelle route fonctionne et que le contrôleur affiche le template Twig.

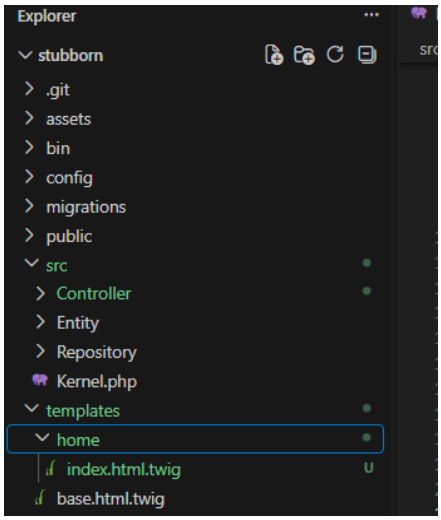
Réalisation pratique

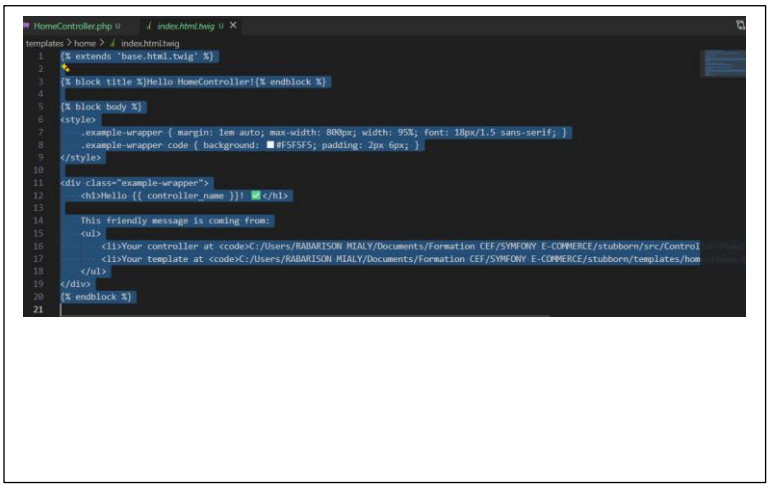
5. Laisser le serveur Symfony en fonctionnement.
6. Ouvrir un navigateur Web.
7. Accéder à http://127.0.0.1:8000/.

Résultat obtenu

La page provisoire « Your application is ready » n'apparaît plus. Elle est remplacée par la page générée par HomeController, affichant notamment « Hello HomeController! ».







Explication

Ce résultat confirme que la route / fonctionne, que HomeController est exécuté et que le template home/index.html.twig est utilisé.

Conclusion

La première page d'accueil fonctionne. Son contenu est encore provisoire et doit être personnalisé.

5.2 Personnalisation de la page d'accueil STUBBORN

Objectif

Personnaliser la page d'accueil du site e-commerce STUBBORN en remplaçant les éléments provisoires générés par Symfony et en préparant une structure conforme aux wireframes fournis dans le devoir.

Présentation

Le cahier des charges prévoit une page d'accueil accessible à la route /, comprenant :

- Le logo officiel STUBBORN.
- Un menu de navigation adapté à l'état de connexion de l'utilisateur.
- Les trois sweatshirts mis en avant par la marque.
- Une courte présentation de la société.

Les wireframes présentent deux versions de la page d'accueil : une pour les visiteurs non connectés et une pour les utilisateurs connectés.

Dans les deux versions, les trois produits sont présentés avant les informations de la société.

Réalisation pratique

5.2.1 Ouverture du template Twig de la page d'accueil

Dans Visual Studio Code :

1. Ouvrir le dossier templates.
2. Ouvrir le sous-dossier home.
3. Sélectionner le fichier index.html.twig.

Ce fichier contient le contenu spécifique de la page d'accueil.

Présentation de Twig

Twig est le moteur de templates utilisé par Symfony. Il permet d'associer du HTML à des instructions de présentation.

- {% ... %} : exécute une instruction Twig.
- {{ ... }} : affiche une valeur.
- {# ... #} : insère un commentaire Twig non affiché dans le navigateur.

5.2.2 Présentation du template commun base.html.twig

Le fichier templates/base.html.twig définit la structure commune des pages du site.

Élément	Fonction
<head>	Contient les informations techniques de la page.
<body>	Contient les éléments visibles.
{% block title %}	Définit le titre de l'onglet du navigateur.
{% block body %}	Définit l'emplacement du contenu propre à chaque page.

Dans home/index.html.twig, l'instruction :

{% extends 'base.html.twig' %}

signifie que la page d'accueil utilise la structure commune définie dans base.html.twig.

Cette organisation évite de répéter l'en-tête et les autres éléments communs dans chaque page.

5.2.3 Modification du titre de la page

Dans templates/home/index.html.twig, le titre provisoire généré par Symfony a été remplacé par :

{% block title %}Stubborn - Boutique de sweatshirts{% endblock %}

Explication technique

Le bloc title définit le texte affiché dans l'onglet du navigateur.

Il ne correspond pas au titre visible dans le contenu de la page.

5.2.4 Personnalisation du fond avec CSS

Ouvrir le fichier :

assets/styles/app.css

Le code provisoire :

```
body {
  background-color: skyblue;
}
```

a été remplacé par :

```
body {
  background-color: white;
}
```

Explication technique

- body : sélectionne le corps de la page.
- background-color : définit la couleur de fond.
- white : applique une couleur blanche.

Résultat obtenu

Le fond de la page d'accueil est blanc, conformément à l'apparence générale des wireframes.

Conclusion

Cette étape a permis de personnaliser le titre et l'apparence générale de la page d'accueil, tout en comprenant le rôle des fichiers Twig et CSS.

5.3 Intégration du logo officiel dans l'en-tête

Objectif

Créer un en-tête commun au site STUBBORN et y intégrer le logo officiel fourni dans les ressources du devoir.

Présentation

Les wireframes montrent le logo STUBBORN dans la partie supérieure gauche de la page, tandis que le menu de navigation apparaît à droite.

L'en-tête est intégré dans base.html.twig afin d'être réutilisé sur les différentes pages du site.

Réalisation pratique

5.3.1 Importation du logo

Le fichier Logo.png, fourni dans les ressources du devoir, a été copié dans :

public/images/Logo.png

Le dossier public contient les ressources accessibles par le navigateur.

Le sous-dossier images regroupe les éléments graphiques du site.

5.3.2 Création de l'en-tête dans base.html.twig

Dans le fichier templates/base.html.twig, la balise <header> a été placée au début du <body>, avant le bloc Twig body.

Le logo est affiché avec :

```
<header>
  <img src="{{ asset('images/Logo.png') }}" alt="Logo Stubborn">
  {# En-tête commun aux pages du site #}
</header>
```

Explication technique

- <header> : définit l'en-tête du site.
- <img> : affiche une image.
- src : indique le chemin de l'image.
- asset() : génère l'URL d'un fichier public dans Symfony.
- alt : fournit une description textuelle de l'image.

Nous écrivons images/Logo.png dans asset(), car le dossier public constitue déjà le point de départ des ressources publiques.

Résultat obtenu

Le logo officiel STUBBORN apparaît en haut à gauche de la page d'accueil après ajustement de CSS

Conclusion

L'intégration du logo permet d'utiliser l'identité graphique fournie avec le devoir et de préparer un en-tête commun à l'ensemble du site.

5.4 Création du menu de navigation

Objectif

Intégrer un menu de navigation dans l'en-tête en respectant les exigences du cahier des charges.

Présentation

Le cahier des charges distingue deux menus.

Utilisateur non connecté :

Accueil — S'inscrire — Se connecter

Utilisateur connecté :

Accueil — Boutique — Panier — Se déconnecter

Le menu doit changer automatiquement selon l'état de connexion.

À ce stade du développement, le système d'authentification n'est pas encore réalisé. De ce fait le menu affiché ici est prévu pour un visiteur non connecté.

Réalisation pratique

5.4.1 Ajout du menu dans base.html.twig prévu pour un visiteur non conecté

```
<nav>
    <a href="/">Accueil</a>
    <a href="/register">S'inscrire</a>
    <a href="/login">Se connecter</a>
</nav>
```

Explication technique

- <nav> : définit une zone de navigation.
- <a> : crée un lien hypertexte.
- href : précise l'adresse de destination.
- / : route de la page d'accueil.
- /register : future route d'inscription.
- /login : future route de connexion.

5.4.2 Structure complète de l'en-tête

Le code actuel de l'en-tête est :

```
<header>
    <div class="identite">
<img src="{{ asset('images/Logo.png') }}" alt="Logo Stubborn">
<p class="slogan">Don't compromise on your look</p>
    </div>

   <nav>
    <a href="/" class="{{ app.request.attributes.get('_route') == 'app_home' ? 'actif' : '' }}">Accueil</a>
    <a href="/register">S'inscrire</a>
    <a href="/login">Se connecter</a>
   </nav>
    {# Menu provisoire pour un utilisateur non connecté. #}

</header>
```

{# Menu provisoire pour un utilisateur non connecté. #}

5.4.3 Préparation du menu dynamique

Le menu actuel est statique : il affiche toujours les mêmes liens.

Lors de la mise en place de Symfony Security, nous utiliserons l'état de connexion de l'utilisateur pour afficher automatiquement le menu approprié.

Les routes /register et /login ne sont pas encore développées.

Résultat obtenu

L'en-tête affiche le logo STUBBORN et les trois liens prévus pour un utilisateur non connecté.

Conclusion

La structure du menu est en place. Son fonctionnement dynamique sera développé lors de la mise en œuvre de l'authentification Symfony.

5.5 Organisation du contenu de la page d'accueil

Objectif

Organiser les différentes sections de la page d'accueil du site STUBBORN conformément aux wireframes du projet.

Présentation

Les wireframes prévoient une organisation de la page d'accueil en trois parties :

1. L'en-tête comprenant le logo et le menu de navigation.
2. Les trois sweatshirts mis en avant.
3. Les informations de la société.

La section de présentation de la société doit donc être placée après les fiches produits.

Réalisation pratique

5.5.1 Organisation du template de la page d'accueil

Le fichier templates/home/index.html.twig a été modifié afin d'organiser les différentes sections à l'intérieur du bloc Twig body.

5.5.2 Organisation de la section des produits

Une section identifiée par la classe produits a été prévue pour regrouper les trois sweatshirts mis en avant : Blackbelt, Pokeball et BornInUsa.

Cette classe permettra d'appliquer une mise en forme commune à la section avec CSS.

5.5.3 Organisation de la section des informations de la société

Une section identifiée par la classe presentation a été placée après celle des produits.

Elle contient le titre « Informations de la société » ainsi qu'un texte provisoire qui sera complété à partir des ressources du devoir.

Lors de la réalisation, deux sections presentation avaient été imbriquées, provoquant l'apparition de deux encadrés. Cette erreur a été corrigée en conservant une seule section.

Résultat obtenu

La page d'accueil respecte désormais l'ordre des éléments prévu dans les wireframes : en-tête, produits mis en avant, puis informations de la société.

Conclusion

Cette étape a permis de structurer le contenu de la page d'accueil conformément aux wireframes et de préparer la présentation des produits et des informations de la société.

5.6 Création des fiches produits sur la page d’accueil

Objectif

Présenter les trois sweatshirts mis en avant par la marque STUBBORN :

Produit	Prix
Blackbelt	29,90 €
Pokeball	45,00 €
BornInUsa	59,90 €

Chaque fiche doit présenter une photographie, le nom du sweatshirt et son prix.

Présentation

La réalisation comprend la préparation des images, l'intégration des trois fiches dans Twig et leur mise en forme avec CSS.

Dans cette première version, les informations sont écrites directement dans le template. L'affichage sera rendu dynamique lors de l'intégration de MySQL.

Réalisation pratique

5.6.1 Création du dossier images

- Le dossier public contient les ressources publiques de l'application.
- Le dossier images permet de regrouper les images utilisées par le site.
- Le fichier ZIP fourni avec le devoir contient les images des dix sweatshirts.
- Les images ont été extraites puis copiées dans public/images.

Les noms des fichiers doivent correspondre exactement aux chemins utilisés dans Twig.

5.6.2 Intégration des trois fiches produits

Les trois sweatshirts mis en avant, **Blackbelt (29,90 €), Pokeball (45,00 €) et BornInUsa (59,90 €)**, ont été intégrés dans le fichier templates/home/index.html.twig.

Ils ont été regroupés dans une même section HTML identifiée par la classe produits, sous le titre « Nos sweatshirts à découvrir ».

Chaque sweatshirt est placé dans un conteneur utilisant la classe commune produit. Ce conteneur regroupe trois éléments : la photographie du sweatshirt, son nom et son prix.

Les photographies sont affichées à l'aide de la balise img et de la fonction Twig asset(), qui permet d'accéder aux images enregistrées dans le dossier public/images.

Les noms des produits sont définis avec la balise h3 et leurs prix avec la balise p.

L'utilisation de la même classe produit pour les trois fiches permet de leur appliquer une mise en forme CSS identique.

```
<section class="produits">
    <h2>Nos sweatshirts à découvrir</h2>

    <div class="produit">
        <img src="{{ asset('images/Blackbelt.jpeg') }}" alt="Sweatshirt Blackbelt">
        <h3>Blackbelt</h3>
        <p>Prix : 29,90 €</p>
    </div>

    <div class="produit">
        <img src="{{ asset('images/Pokeball.jpeg') }}" alt="Sweatshirt Pokeball">
        <h3>Pokeball</h3>
        <p>Prix : 45,00 €</p>
    </div>

    <div class="produit">
        <img src="{{ asset('images/BornInUsa.jpeg') }}" alt="Sweatshirt BornInUsa">
        <h3>BornInUsa</h3>
        <p>Prix : 59,90 €</p>
    </div>

</section>
```

5.7 Finalisation de la page d'accueil pour un utilisateur non connecté

La page d'accueil a été finalisée en respectant le cahier des charges et le wireframe fourni.

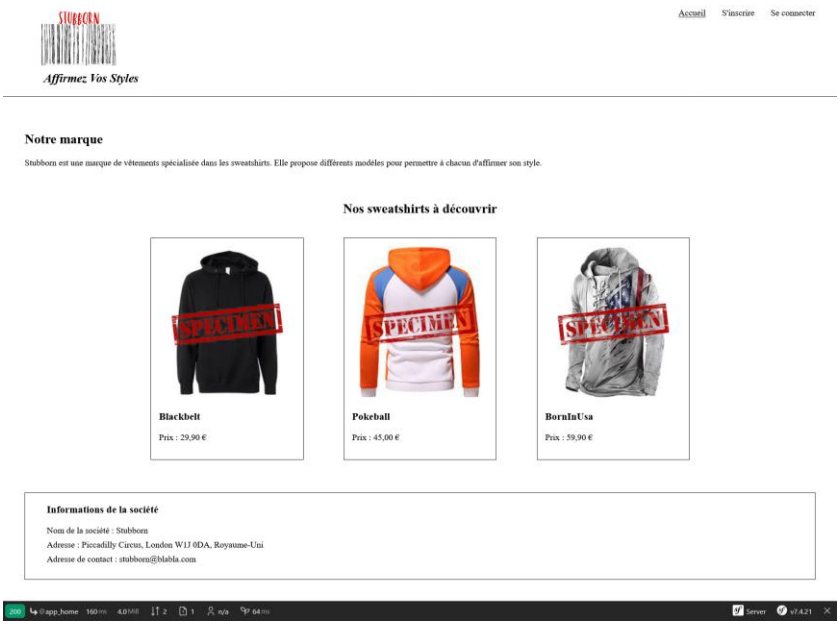
Elle comprend le logo Stubborn accompagné de son slogan « Don't compromise on your look », ainsi qu'un menu de navigation composé des liens Accueil, S'inscrire et Se connecter.

Trois sweatshirts sont présentés avec leurs images, leurs noms et leurs prix : Blackbelt, Pokeball et BornInUsa.

Une section présente également la société Stubborn, ses coordonnées et une courte description de son activité.

Le lien Accueil est automatiquement souligné lorsque l'utilisateur consulte cette page, grâce à la détection de la route active dans Twig.

À ce stade, la page correspond à la version destinée à un utilisateur non connecté. Les fonctionnalités nécessitant une authentification seront développées dans les étapes suivantes.



6 Gestion des utilisateurs et authentification

La gestion des utilisateurs et l'authentification constituent une étape importante du développement du site e-commerce **STUBBORN**.

L'objectif est de permettre aux visiteurs de créer un compte personnel, de se connecter de manière sécurisée et d'accéder aux fonctionnalités réservées aux utilisateurs authentifiés.

Cette partie du développement comprend notamment la création de la base de données, l'enregistrement des informations des utilisateurs, la mise en place d'un formulaire d'inscription avec confirmation par e-mail, ainsi que la gestion de la connexion et de la déconnexion.

Symfony propose des outils permettant de gérer ces fonctionnalités, notamment **Doctrine** pour les échanges avec la base de données et le composant **Security** pour l'authentification et la gestion des accès.

Les différentes étapes de mise en œuvre sont présentées dans les sections suivantes, en commençant par la configuration de la base de données MySQL.

6.1 Configuration de la base de données

X

Objectif

Cette étape consiste à créer une base de données MySQL pour le site e-commerce STUBBORN et à établir la connexion entre cette base et l'application Symfony.

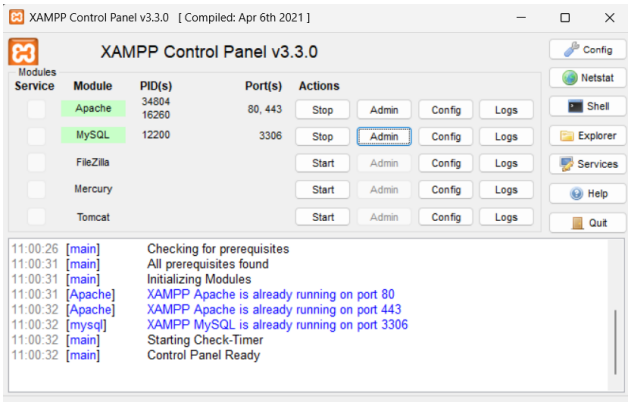
6.1.1 Démarrage des services avec XAMPP

Le logiciel XAMPP est utilisé pour faire fonctionner les services nécessaires au développement local.

Dans le panneau de contrôle XAMPP, les services suivants ont été démarrés :

- **Apache** : serveur web local.
- **MySQL** : système de gestion de base de données.

L'interface **phpMyAdmin** a ensuite été ouverte à partir du bouton « Admin » situé sur la ligne MySQL de XAMPP.



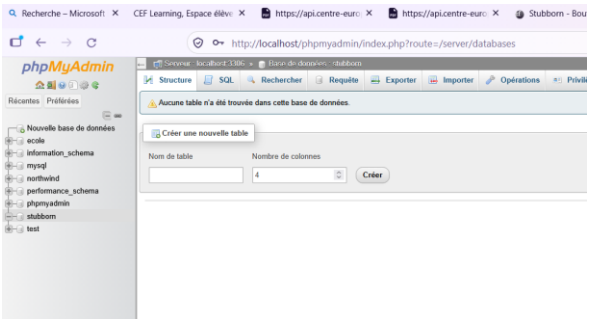
6.1.2 Création de la base de données

Dans phpMyAdmin, une nouvelle base de données a été créée avec les paramètres suivants :

Paramètre	Valeur
Nom de la base de données	stubborn
Système de gestion	MySQL
Interclassement	utf8mb4_unicode_ci

L'interclassement utf8mb4_unicode_ci permet de prendre en charge les caractères accentués et les caractères spéciaux.

Après validation, la base de données stubborn est apparue dans le menu de gauche de phpMyAdmin.



6.1.3 Configuration de la connexion dans Symfony

Le fichier .env, situé à la racine du projet Symfony, contient les paramètres de connexion à la base de données.

La configuration initiale utilisait PostgreSQL :

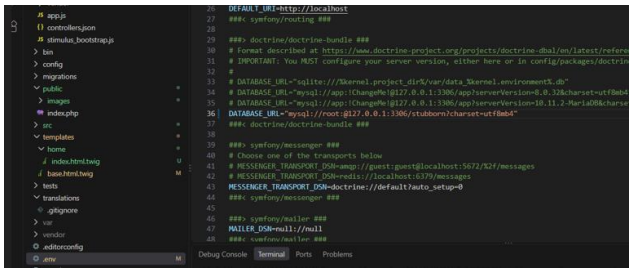
DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=16&charset=utf8"

Cette ligne a été remplacée afin d'utiliser la base MySQL créée dans XAMPP :

DATABASE_URL="mysql://root:@127.0.0.1:3306/stubborn?charset=utf8mb4"

Les différents paramètres correspondent à :

- mysql : système de gestion de base de données utilisé.
- root : nom de l'utilisateur MySQL.
- 127.0.0.1 : adresse du serveur local.
- 3306 : port utilisé par MySQL.
- stubborn : nom de la base de données.
- charset=utf8mb4 : encodage des caractères.



Cette configuration correspond à l'environnement de développement local, avec un utilisateur MySQL root sans mot de passe.

6.1.4 Vérification de la connexion

Pour vérifier la communication entre Symfony et MySQL, la commande suivante a été exécutée dans le terminal de VS Code, depuis le dossier du projet :

php bin/console doctrine:query:sql "SELECT 1"

Résultat obtenu :

La commande s'est exécutée sans erreur et a retourné la valeur 1.

Ce résultat confirme que Symfony peut se connecter à MySQL et exécuter une requête SQL.



6.1.5 Résultat de cette étape

La base de données stubborn a été créée avec succès dans MySQL. Le fichier .env du projet Symfony a été configuré et la connexion a été vérifiée par une requête SQL.

L'application dispose désormais d'une connexion fonctionnelle à sa base de données, ce qui permet de poursuivre le développement avec la création de l'entité utilisateur et la mise en place de l'authentification.

6.2 Création de l'entité User

Objectif

Cette étape consiste à créer l'entité User, qui permettra de représenter les utilisateurs du site e-commerce STUBBORN et d'enregistrer leurs informations dans la base de données MySQL.

Selon le cahier des charges, un utilisateur client est caractérisé par quatre informations : un nom (name), une adresse e-mail (email), un mot de passe (password) et une adresse de livraison (delivery_address).

L'entité User constitue donc la base nécessaire à la mise en place des fonctionnalités d'inscription et d'authentification.

6.2.1 Génération de l'entité User avec Symfony

La création de l'entité a été effectuée depuis le terminal de VS Code, à la racine du projet STUBBORN, à l'aide de la commande suivante :

php bin/console make:user

Symfony a ensuite demandé plusieurs informations pour configurer l'utilisateur :

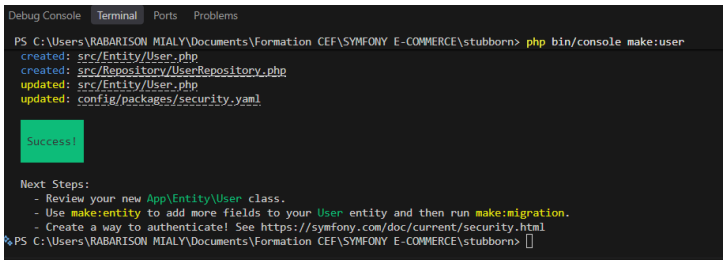
Question posée par Symfony	**Réponse choisie**
Nom de la classe utilisateur	User
Enregistrement des utilisateurs avec Doctrine	Yes
Identifiant unique de l'utilisateur	Email
Gestion et hachage des mots de passe	Yes

Ces choix permettent d'identifier chaque utilisateur par son adresse e-mail et de prévoir le stockage sécurisé de son mot de passe.

À l'issue de cette commande, Symfony a confirmé la création de l'entité avec le message Success!.

Les fichiers suivants ont été créés ou modifiés :

- src/Entity/User.php : définition de l'entité utilisateur.
- src/Repository/UserRepository.php : gestion des recherches d'utilisateurs dans la base de données.
- config/packages/security.yaml : configuration de la sécurité de l'application.



6.2.2 Ajout des propriétés demandées par le cahier des charges

L'entité générée automatiquement par Symfony contenait déjà les propriétés id, email, roles et password.

Cependant, le cahier des charges de STUBBORN prévoit également le nom du client et son adresse de livraison.

Pour ajouter ces deux propriétés, la commande suivante a été exécutée :

php bin/console make:entity User

Symfony a détecté que l'entité existait déjà et a proposé d'y ajouter de nouveaux champs.

Les propriétés suivantes ont été configurées :

Propriété PHP	Type	Longueur	Nullable
name	string	255	Non
deliveryAddress	string	255	Non

Le champ name permet de conserver le nom du client.

Le champ deliveryAddress permet d'enregistrer son adresse de livraison. Dans la base de données, Doctrine utilise le nom de colonne delivery_address.

Les deux propriétés ont été définies comme obligatoires au niveau de la base de données.

La commande s'est terminée avec le message Success!, confirmant la mise à jour du fichier src/Entity/User.php.

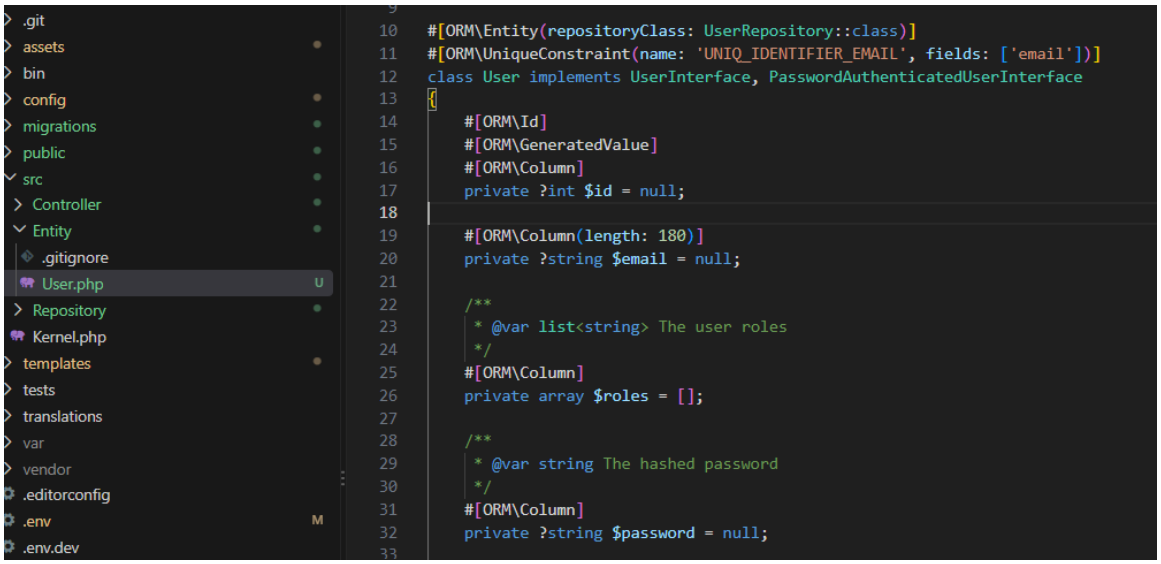


Figure 6.1 — Structure de l'entité User : propriétés des utilisateurs du site STUBBORN.

6.2.3 Structure de l'entité User

Après ces modifications, l'entité User contient les propriétés suivantes :

Propriété	Description
id	Identifiant unique généré automatiquement.
name	Nom du client.
email	Adresse e-mail unique utilisée pour l'identification.
password	Mot de passe destiné à être enregistré sous forme hachée.
deliveryAddress	Adresse de livraison du client.
roles	Liste des rôles permettant de gérer les droits d'accès.

Les propriétés id et roles ont été ajoutées automatiquement par Symfony pour répondre aux besoins techniques de l'application.

L'entité implémente également les interfaces UserInterface et PasswordAuthenticatedUserInterface, utilisées par le système de sécurité de Symfony.

À ce stade, la structure de l'utilisateur est définie, mais les formulaires d'inscription et de connexion ne sont pas encore développés.

6.2.4 Création de la migration Doctrine

Après la création et la modification de l'entité, une migration a été générée pour préparer la création des tables dans MySQL.

La commande utilisée est :

php bin/console make:migration

Symfony a généré le fichier suivant dans le dossier migrations :

Version20261023110041.php

Le contenu de cette migration a été vérifié avant son exécution.

Elle prévoit notamment la création de la table user avec les colonnes :

- id
- email
- roles
- password

- name
- delivery_address

Une contrainte d'unicité est également prévue sur la colonne email, afin d'empêcher l'enregistrement de plusieurs utilisateurs avec la même adresse e-mail.

La migration prévoit aussi la création de la table messenger_messages, liée au composant Symfony Messenger.

6.2.5 Exécution de la migration

Après vérification du fichier généré, la migration a été exécutée à l'aide de la commande :

php bin/console doctrine:migrations:migrate

Symfony a affiché un avertissement indiquant que cette opération pouvait modifier la structure de la base de données.

L'exécution a été confirmée en conservant la réponse yes.

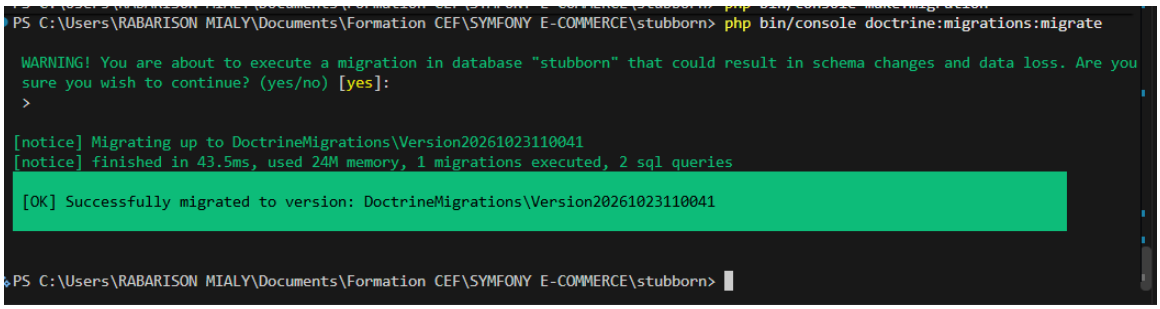
Le terminal a ensuite affiché le message :

[notice] Migrating up to DoctrineMigrations\Version20261023110041

[notice] finished in 43.5ms, used 24M memory, 1 migrations executed, 2 sql queries

[OK] Successfully migrated to version: DoctrineMigrations\Version20261023110041

Ce résultat confirme que la migration a été exécutée avec succès dans la base de données stubborn.



6.2.6 Vérification des tables dans phpMyAdmin

Une vérification a ensuite été réalisée dans phpMyAdmin, en ouvrant la base de données stubborn.

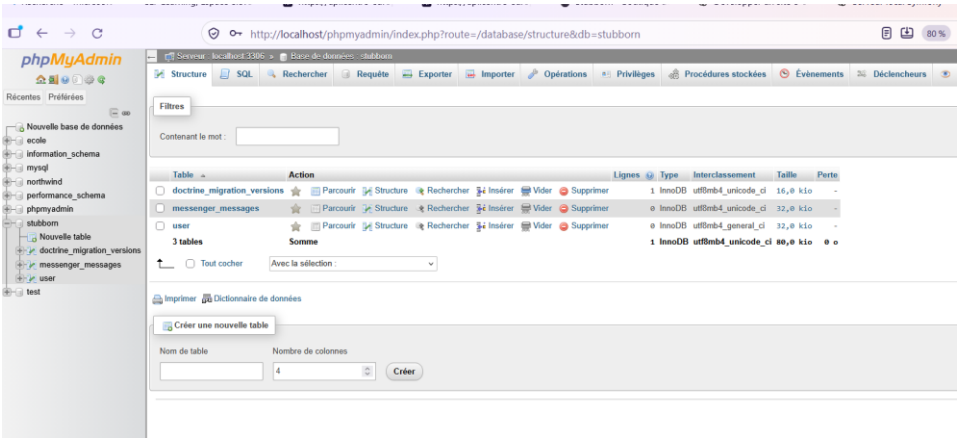
Trois tables étaient présentes :

Table	Fonction

user	Stockage des informations des utilisateurs.
messenger_messages	Gestion des messages par Symfony Messenger.
doctrine_migration_versions	Suivi des migrations Doctrine exécutées.

La présence de ces trois tables confirme que la migration a bien été appliquée.

La table user ne contient encore aucun enregistrement, ce qui est normal puisque le formulaire d'inscription n'a pas encore été créé.



6.2.7 Résultat de cette étape

L'entité User a été créée avec succès et complétée avec les informations demandées par le cahier des charges de STUBBORN.

La migration Doctrine a été générée, vérifiée puis exécutée. La table user est désormais présente dans la base de données MySQL stubborn.

Cette étape prépare la gestion des comptes clients et permet de poursuivre le développement avec la création du formulaire d'inscription.

6.3 Création du formulaire d'inscription

Le formulaire d'inscription a été créé avec Symfony afin de permettre aux nouveaux utilisateurs de créer un compte.

Les informations sont enregistrées dans la base de données. Le mot de passe est sécurisé grâce au système de hachage de Symfony.

Figure 6.2 — Formulaire d'inscription STUBBORN.

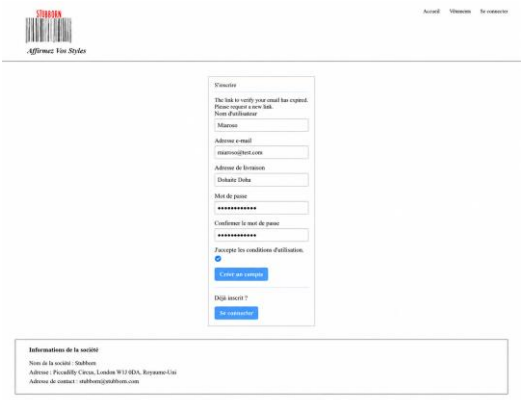
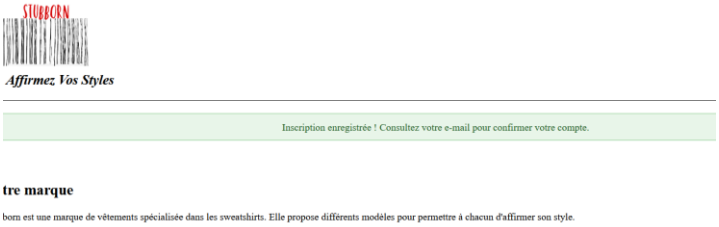


Figure 6.3 — Message de demande de confirmation de compte.



6.4 Vérification de l'adresse e-mail et activation du compte

Une vérification de l'adresse e-mail a été mise en place avec Symfony et Mailtrap.

Après son inscription, l'utilisateur reçoit un e-mail contenant un lien de confirmation. Il doit valider ce lien pour activer son compte.

Une vérification empêche également la connexion tant que l'adresse e-mail n'a pas été confirmée.

Figure 6.4 — E-mail de vérification reçu dans Mailtrap.

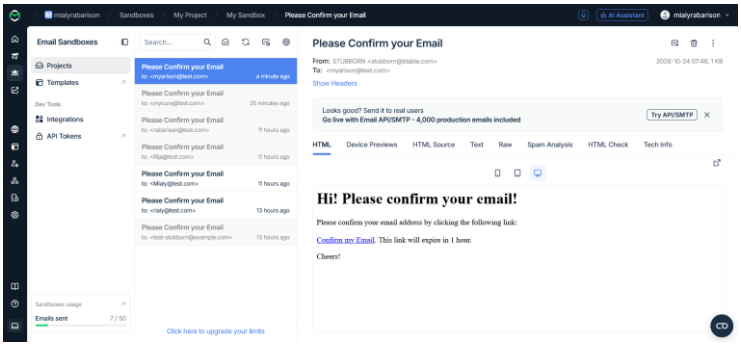


Figure 6.5 — Message de confirmation après activation du compte.

STUBBORN
ffirmez Vos Styles

Votre adresse e-mail a été confirmée.

6.5 Connexion et déconnexion des utilisateurs

Le formulaire de connexion permet aux utilisateurs enregistrés de s'authentifier avec leur adresse e-mail et leur mot de passe.

Après validation de son adresse e-mail, l'utilisateur peut accéder à son espace connecté. Le lien « Se déconnecter » permet de terminer sa session.

Figure 6.6 — Formulaire de connexion.



6.6 Adaptation de l'interface selon l'état de connexion

Les fichiers templates/home/index.html.twig et templates/base.html.twig ont été adaptés pour afficher des contenus différents selon l'état de connexion.

Le visiteur dispose des liens Accueil, S'inscrire et Se connecter.

Le client connecté dispose des liens Accueil, Boutique, Panier et Se déconnecter. Sa page d'accueil présente également trois sweatshirts avec un bouton « VOIR » pour chaque produit.

La mise en forme a été réalisée dans assets/styles/app.css.

Figure 6.7 — Page d'accueil du visiteur non connecté.

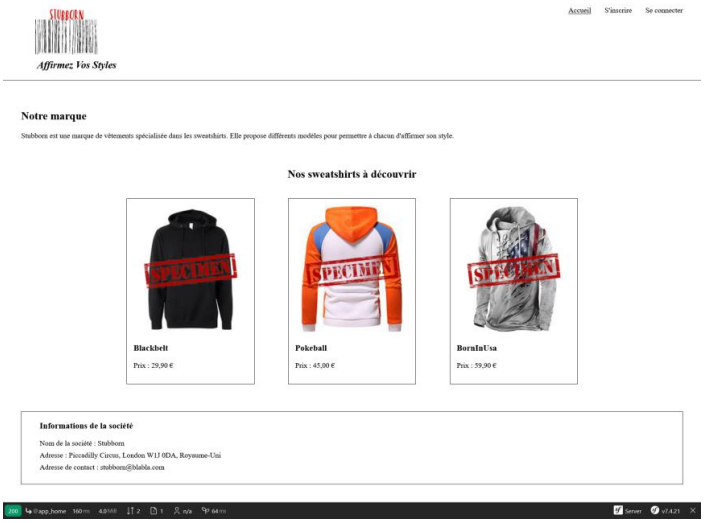
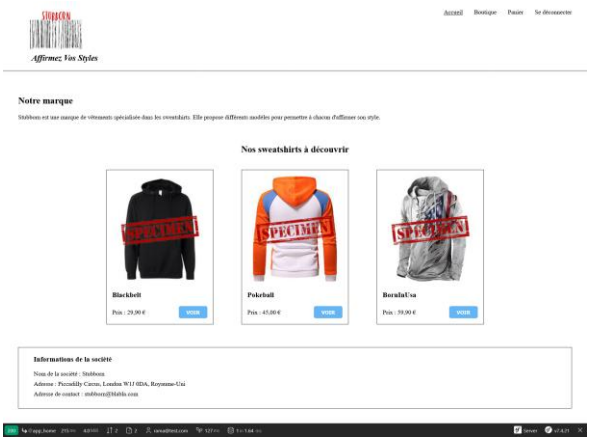


Figure 6.8 — Page d'accueil du client connecté avec le menu et les boutons VOIR.



7 Création de la boutique et des fiches produits

7.1 Création du contrôleur et des routes /products et /product/{id}

Le contrôleur ProductController a été créé à l'aide de la commande Symfony suivante :

php bin/console make:controller ProductController

Cette commande a généré les fichiers :

- src/Controller/ProductController.php
- templates/product/index.html.twig

Le contrôleur a ensuite été modifié pour définir les deux routes demandées dans le cahier des charges :

- **/products** : affichage de la page Boutique contenant la liste des sweatshirts.
- **/product/{id}** : accès à la fiche individuelle d'un sweatshirt grâce à son identifiant.

La fiche individuelle affiche provisoirement un message contenant le numéro du produit. Son contenu sera développé ultérieurement.

```
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn> php bin/console make:controller ProductController

  Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
  >

  created: src/Controller/ProductController.php
  created: templates/product/index.html.twig

  Success!

  Next: Open your new controller class and add some pages!
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn>
```

7.2 Création des données des 10 sweatshirts : noms, prix, images, tailles et stocks

L'entité Product a été créée avec la commande Symfony php bin/console make:entity Product.

Elle contient les propriétés suivantes : identifiant, nom, prix, image et stocks pour les tailles XS, S, M, L et XL. Les types des propriétés ont été vérifiés et corrigés dans src/Entity/Product.php.

Figure 7.1 — Création de l'entité Product dans le terminal.

```
  Next: Open your new controller class and add some pages!
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn> php bin/console make:entity Product
  created: src/Entity/Product.php
  created: src/Repository/ProductRepository.php

  Entity generated! Now let's add some fields!
  You can always add more fields later manually or by re-running this command.

  New property name (press <return> to stop adding fields):
  >
```

Une migration a ensuite été générée avec php bin/console make:migration, puis exécutée avec php bin/console doctrine:migrations:migrate pour créer la table product dans MariaDB.

Figure 7.2 — Génération et exécution de la migration Symfony.

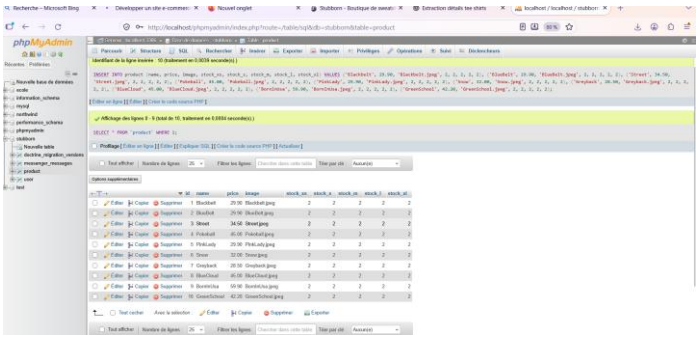
```
    }
}
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn> php bin/console make:migration
  created: migrations/Version20261024103809.php

  Success!

  Review the new migration then run it with php bin/console doctrine:migrations:migrate
  See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html
PS C:\Users\RABARISON MIALY\Documents\Formation CEF\SYMFONY E-COMMERCE\stubborn>
```

Les 10 sweatshirts ont ensuite été insérés dans la table product à l'aide d'une requête SQL dans phpMyAdmin, avec leurs noms, prix, images et un stock de 2 exemplaires par taille.

Figure 7.3 — Enregistrement des 10 sweatshirts et vérification des stocks dans phpMyAdmin.



7.3 Création de la page Boutique avec les 10 produits, conformément au wireframe

La page Boutique a été réalisée avec le fichier templates/product/index.html.twig et est accessible à l'adresse /products.

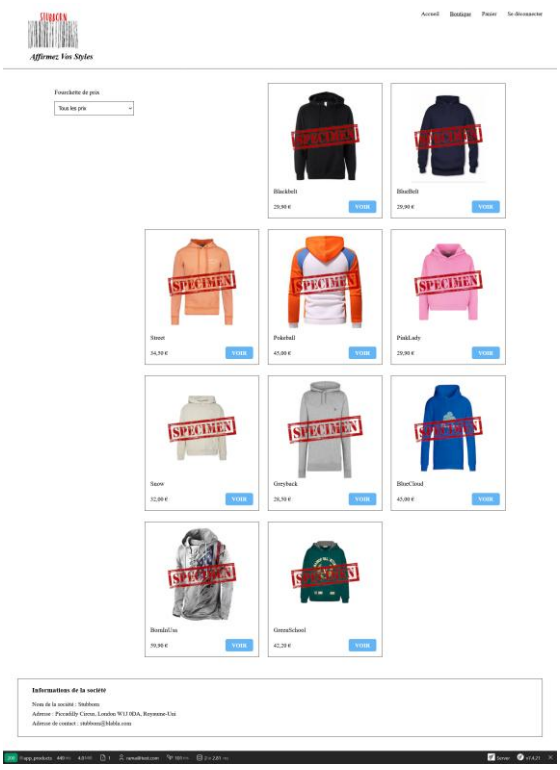
Le contrôleur ProductController.php utilise ProductRepository pour récupérer les 10 sweatshirts enregistrés dans la base de données MariaDB et les transmettre au template Twig.

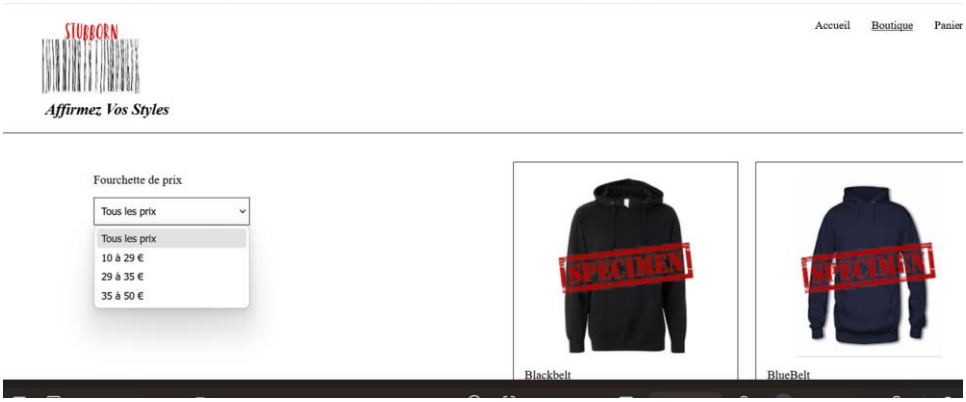
Chaque sweatshirt est présenté avec son image, son nom, son prix et un bouton **VOIR** permettant d'accéder à sa fiche individuelle.

La mise en forme a été réalisée dans assets/styles/app.css, conformément au wireframe fourni : filtre de prix à gauche, cartes de produits encadrées, disposition en grille, prix à gauche et boutons VOIR à droite.

Le lien **Boutique** du menu de navigation a également été configuré pour accéder à la route /products et être souligné lorsque cette page est active.

Figure 7.4 — Affichage de la page Boutique avec les 10 sweatshirts et la mise en forme CSS conforme au wireframe.





7.4 Mise en place du filtre par fourchette de prix

Un système de filtrage par prix a été intégré à la page Boutique, conformément au cahier des charges.

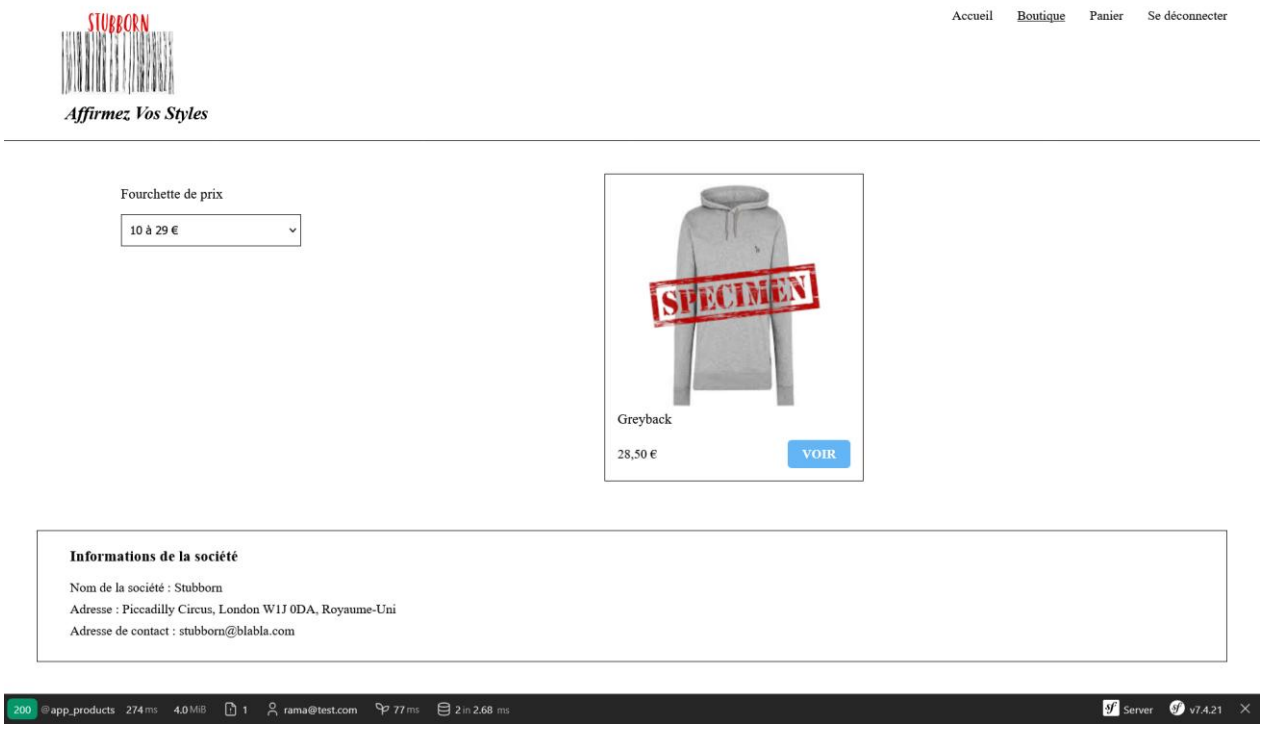
Le fichier ProductController.php a été modifié pour récupérer la fourchette sélectionnée à l'aide de l'objet Request et filtrer les produits selon leur prix.

Dans templates/product/index.html.twig, une liste déroulante permet de sélectionner les fourchettes **10 à 29 €**, **29 à 35 €** et **35 à 50 €**. La page se recharge automatiquement après chaque sélection et affiche uniquement les sweatshirts correspondants.

L'option **Tous les prix** permet de réafficher les 10 produits.

Les tests des différentes fourchettes ont confirmé le bon fonctionnement du filtrage.

Figure 7.5 — Affichage des sweatshirts après sélection d'une fourchette de prix.



7.5 Création de la fiche individuelle d'un sweatshirt

7.5.1 Création de la route de consultation d'un produit

Dans le fichier src/Controller/ProductController.php, ajout d'une méthode show() associée à la route /product/{id}.

Cette méthode permet de récupérer un sweatshirt dans la base de données MariaDB à partir de son identifiant grâce au ProductRepository.

Si le produit n'existe pas, une erreur 404 est retournée. Sinon, les informations du produit sont transmises au template Twig product/show.html.twig.

7.5.2 Création du template de la fiche produit

Création du fichier templates/product/show.html.twig, qui hérite du template commun base.html.twig.

La fiche affiche les informations du sweatshirt sélectionné :

- Son image ;
- Son nom ;
- Son prix ;
- Les cinq tailles disponibles : XS, S, M, L et XL ;
- Un bouton « AJOUTER AU PANIER ».

Les informations affichées proviennent de la base de données. Le prix est présenté en euros avec deux décimales.

Le bouton d'ajout au panier est présent dans l'interface, mais sa fonctionnalité sera développée lors de l'étape consacrée au panier.

7.5.3 Mise en forme de la fiche produit

Ajout des règles CSS dans le fichier assets/styles/app.css, dans une section dédiée à la fiche individuelle du sweatshirt.

La mise en page respecte l'organisation du wireframe fourni dans le cahier des charges :

- Un grand cadre délimitant la fiche produit ;
- Une grande image du sweatshirt à gauche ;
- Le nom, le prix et la sélection de taille à droite ;
- Les cinq tailles visibles dans une liste ;
- Un bouton « AJOUTER AU PANIER » blanc, avec une bordure noire.

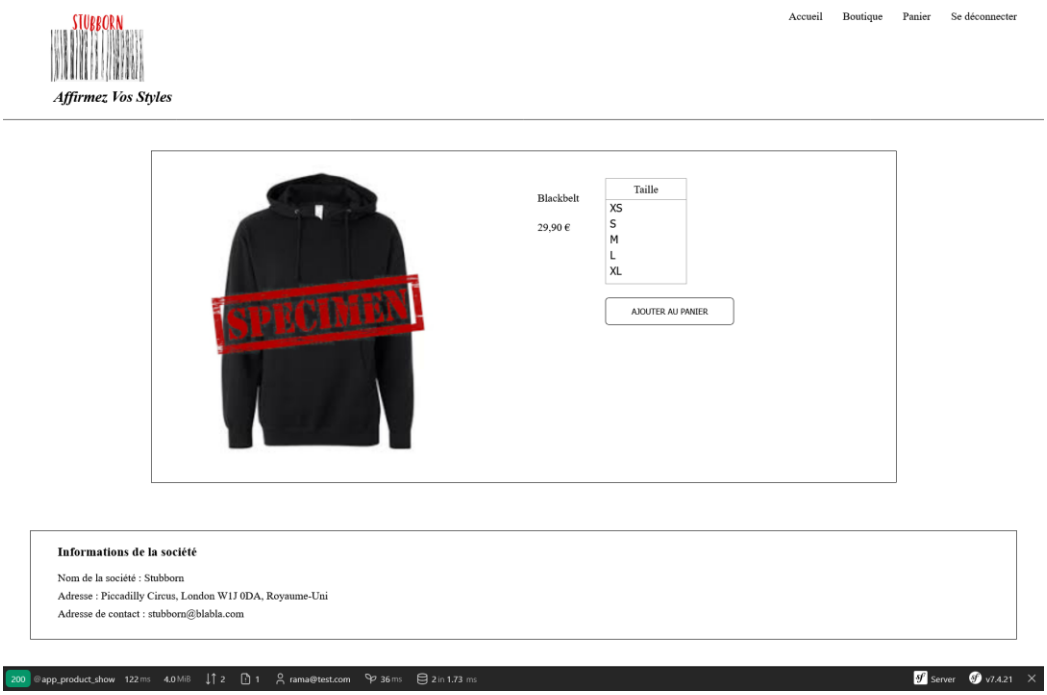
Une règle CSS spécifique permet également de redresser l'image du sweatshirt Greyback.

Des règles d'adaptation sont prévues pour les écrans de plus petite taille.

7.5.4 Vérification de la fiche individuelle

La consultation d'un sweatshirt s'effectue depuis la page Boutique, à l'aide du bouton « VOIR ».

Chaque bouton dirige vers la route /product/{id} correspondant au sweatshirt sélectionné.



8 Création du panier STUBBORN

8.1 Mise en place du contrôleur du panier

Après la réalisation de la page de détail des sweatshirts, nous avons développé la fonctionnalité permettant aux utilisateurs d'ajouter des produits à leur panier.

Le fichier src/Controller/CartController.php a été créé pour gérer les opérations liées au panier.

Le stockage temporaire des articles a été réalisé à l'aide des **sessions Symfony**, permettant de conserver les produits sélectionnés pendant la navigation de l'utilisateur.

Chaque article du panier est identifié par son produit et sa taille. Les informations enregistrées comprennent l'identifiant du produit, la taille sélectionnée et la quantité.

La route /cart/add/{id}, accessible par la méthode POST, permet d'ajouter un sweatshirt au panier.

Avant l'ajout, le contrôleur vérifie l'existence du produit dans la base de données, la validité de la taille sélectionnée et la disponibilité du stock correspondant.

Un jeton CSRF est également utilisé pour sécuriser le formulaire.

8.2 Création de la page d'affichage du panier

Le fichier templates/cart/index.html.twig a été créé pour afficher les articles ajoutés au panier.

La route /cart permet d'accéder à cette page.

Le contrôleur récupère les articles enregistrés dans la session, puis utilise ProductRepository pour retrouver les informations des sweatshirts dans la base de données MariaDB.

Pour chaque article, la page affiche :

- L'image du sweatshirt.
- Son nom et son prix.
- La taille sélectionnée.
- La quantité commandée.
- Un bouton « RETIRER DU PANIER ».

Le montant total est calculé à partir des prix et des quantités des articles.

Lorsque le panier est vide, le message « Votre panier est vide. » est affiché.

8.3 Mise en place de la suppression des articles

La fonctionnalité de suppression a été ajoutée dans CartController.php à l'aide de la route /cart/remove/{cle}, accessible par la méthode POST.

Un formulaire de suppression a été intégré à chaque article dans cart/index.html.twig.

Lorsqu'un utilisateur clique sur « RETIRER DU PANIER », l'article correspondant au produit et à sa taille est supprimé de la session.

Le panier est ensuite actualisé et le montant total recalculé.

Le formulaire est protégé par un jeton CSRF.

8.4 Mise en forme de la page Panier

Le fichier assets/styles/app.css a été modifié afin d'adapter la présentation du panier au wireframe fourni dans le cahier des charges.

Les images, noms, prix, tailles et quantités des sweatshirts ont été organisés pour faciliter la lecture.

Le montant total et les boutons d'action ont été positionnés dans la partie inférieure de la page.

Le bouton « CONTINUER MES ACHATS » permet de revenir au catalogue des produits.

Le bouton « FINALISER MA COMMANDE » a été prévu pour accéder au paiement Stripe.

8.5 Vérification du fonctionnement du panier

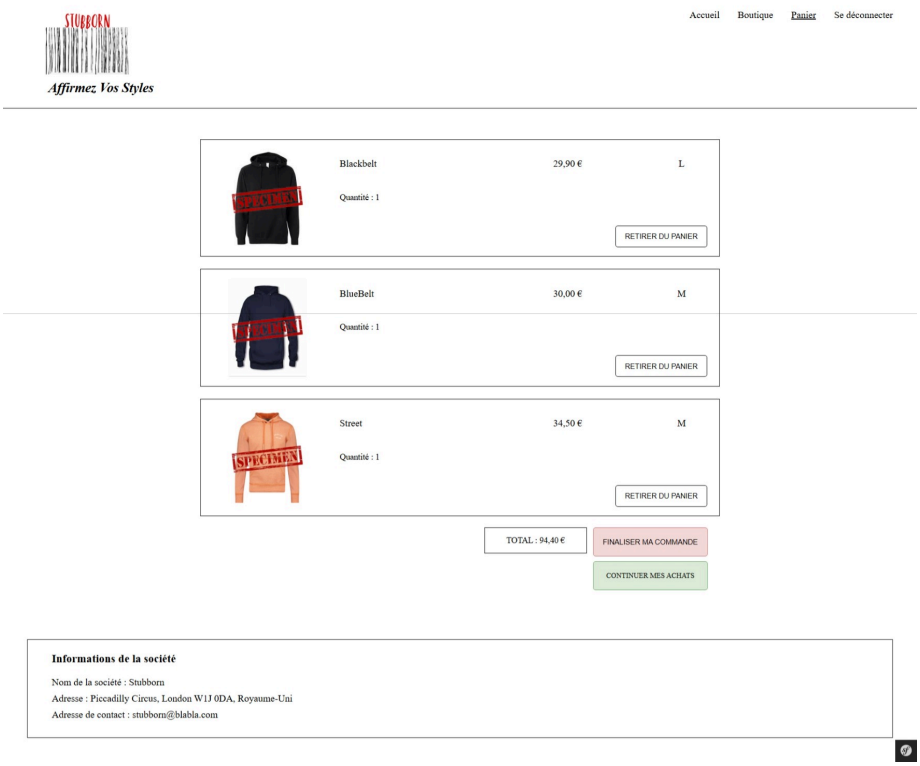
Le fonctionnement du panier a été vérifié dans le navigateur à l'adresse :

http://127.0.0.1:8000/cart

Les tests ont porté sur l'ajout de plusieurs sweatshirts, la sélection de différentes tailles, l'augmentation des quantités, le calcul du total et la suppression des articles.

Les vérifications ont également permis de confirmer l'affichage du message prévu lorsque le panier est vide.

Figure 8.1 — Page Panier STUBBORN présentant les sweatshirts sélectionnés, leurs tailles, leurs quantités et le montant total de la commande.



9 Installation et configuration de Stripe

9.1 Installation de la bibliothèque Stripe

Afin de permettre le paiement des commandes sur STUBBORN, nous avons intégré Stripe Checkout en environnement de test.

Dans le terminal de Visual Studio Code, à la racine du projet Symfony, nous avons exécuté la commande suivante :

composer require stripe/stripe-php

Cette commande a permis d'installer la bibliothèque PHP officielle de Stripe et d'ajouter la dépendance au projet.

Les fichiers composer.json et composer.lock ont été mis à jour automatiquement.

9.2 Création du service Stripe

Le fichier suivant a été créé :

src/Service/StripeService.php

Ce service permet de centraliser les opérations liées au paiement Stripe.

La classe StripeService utilise StripeClient pour communiquer avec l'API Stripe.

La méthode createCheckoutSession() a été développée afin de préparer les articles du panier et de créer une session de paiement.

Pour chaque sweatshirt, le service transmet son nom, sa taille, son prix et la quantité commandée.

Les prix sont convertis en centimes, conformément au format attendu par Stripe.

Une seconde méthode, verifierPaiement(), a ensuite été ajoutée pour vérifier directement auprès de Stripe qu'un paiement a bien été effectué.

9.3 Configuration du service dans Symfony

Le fichier config/services.yaml a été modifié pour transmettre la clé secrète Stripe au constructeur de StripeService.

La configuration suivante a été ajoutée :

```
# Configuration du service Stripe pour les paiements de test

App\Service\StripeService:

  arguments:

    $stripeSecretKey: '%env(STRIPE_SECRET_KEY)%'
```

La clé API est ainsi récupérée depuis une variable d'environnement, sans être écrite directement dans le code PHP.

Le fichier .env.local a été utilisé pour conserver la clé de test localement.

Cette clé confidentielle ne doit pas apparaître dans la documentation ni être publiée dans le dépôt Git.

9.4 Vérification de la configuration Symfony

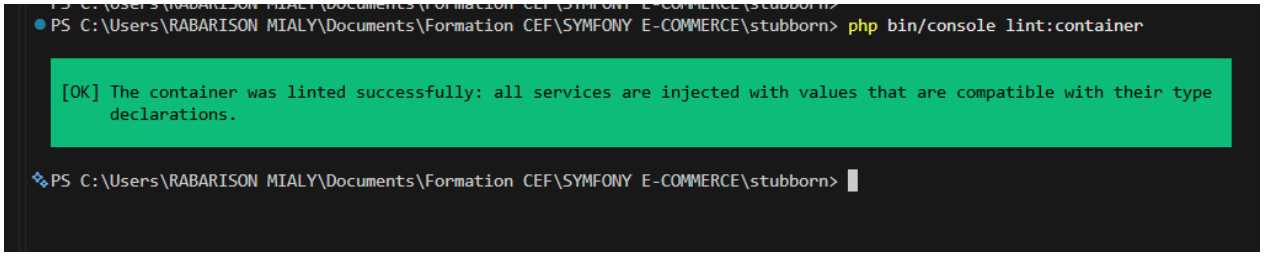
La configuration des services a été vérifiée dans le terminal avec la commande :

```
php bin/console lint:container
```

Symfony a confirmé que la configuration du conteneur de services était valide.

Cette vérification a permis de poursuivre l'intégration du paiement Stripe.

Figure 9.1 — Vérification de la configuration des services Symfony après l'intégration du service Stripe.



10 Installation de Stripe CLI et création du Sandbox de test

10.1 Installation de Stripe CLI

Afin de réaliser des paiements fictifs sans utiliser de véritables transactions bancaires, nous avons mis en place un environnement de test Stripe Sandbox.

Pour cela, nous avons utilisé **Stripe CLI**, l'outil en ligne de commande de Stripe.

La version Windows de Stripe CLI a été téléchargée depuis le dépôt officiel GitHub.

Le fichier ZIP téléchargé a été extrait et l'exécutable stripe.exe a été placé dans le dossier suivant :

C:\stripe-cli\stripe.exe

L'installation a été vérifiée dans le terminal avec la commande :

C:\stripe-cli\stripe.exe --version

Le terminal a confirmé la version installée : **Stripe CLI 1.55.1**.

Figure 10.1 — Vérification de l'installation de Stripe CLI dans le terminal.



10.2 Création de l'environnement Stripe Sandbox

Après l'installation de Stripe CLI, un environnement de test temporaire a été créé à l'aide de la commande :

C:\stripe-cli\stripe.exe sandbox create --email ton-email@example.com

Cette commande a permis de créer un Sandbox Stripe sans ouvrir de compte marchand classique.

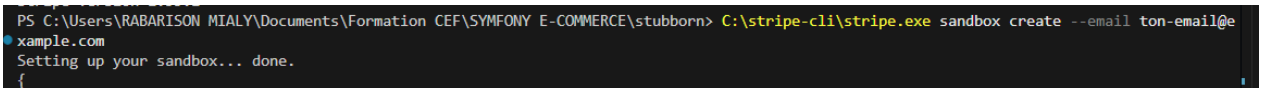
Le terminal a affiché le message confirmant la création de l'environnement de test.

Stripe a également généré plusieurs informations de configuration, notamment :

- Une clé secrète de test.
- Une clé publique de test.
- Un identifiant de compte Sandbox.
- Un lien permettant de réclamer le Sandbox.

L'environnement créé était temporaire, avec une date d'expiration annoncée au **1er novembre 2026**.

Figure 10.2 — Création du Sandbox Stripe et informations de l'environnement de test.



Précaution : les clés API et le lien de réclamation du Sandbox doivent être masqués sur la capture d'écran.

10.3 Configuration de la clé API de test

La clé secrète générée par le Sandbox a été ajoutée au fichier :

.env.local

Elle est associée à la variable d'environnement STRIPE_SECRET_KEY.

Le fichier config/services.yaml, configuré précédemment, permet au service StripeService de récupérer automatiquement cette variable.

Cette organisation évite d'inscrire directement la clé confidentielle dans le code PHP du projet.

Après la configuration, le cache Symfony a été vidé avec la commande :

php bin/console cache:clear

Cette opération a permis de poursuivre les essais avec la configuration actualisée.

10.4 Vérification de la communication avec Stripe

Un premier essai de paiement a permis de vérifier que l'application Symfony communiquait avec l'API Stripe.

Lors de ce test, Stripe a signalé que le paramètre payment_method_types n'était plus accepté pour la création d'une session Checkout dans cet environnement.

Le fichier src/Service/StripeService.php a donc été corrigé en supprimant ce paramètre.

Les moyens de paiement sont désormais déterminés par la configuration Stripe.

Cette correction a permis de poursuivre l'intégration du paiement.

La vérification complète du parcours d'achat est présentée dans les étapes suivantes.

11 Intégration du paiement Stripe au panier STUBBORN

11.1 Intégration du bouton de paiement Stripe

Après la configuration du service Stripe et la création de l'environnement Sandbox, nous avons relié le panier STUBBORN au système de paiement Stripe Checkout.

Le fichier templates/cart/index.html.twig a été modifié pour intégrer un formulaire de paiement au bouton « FINALISER MA COMMANDE ».

Ce formulaire utilise la méthode POST et appelle la route app_cart_checkout.

Un jeton CSRF a été ajouté pour protéger l'envoi du formulaire.

L'attribut data-turbo="false" a également été ajouté afin de permettre au navigateur de gérer directement la redirection vers Stripe Checkout.

11.2 Création de la session de paiement Stripe

Le fichier src/Controller/CartController.php a été complété avec une méthode checkout() associée à la route /cart/checkout.

Cette méthode récupère les articles du panier enregistrés dans la session Symfony.

Avant de lancer le paiement, elle vérifie les produits, les tailles, les quantités et les stocks disponibles.

Les articles sont ensuite transmis au service StripeService, qui crée une session Stripe Checkout.

La session contient les noms des sweatshirts, leurs tailles, leurs prix, leurs quantités et les adresses de retour après paiement ou annulation.

Le client est ensuite redirigé vers la page sécurisée de Stripe.

11.3 Réalisation d'un paiement fictif

Le fonctionnement du paiement a été testé depuis le panier STUBBORN.

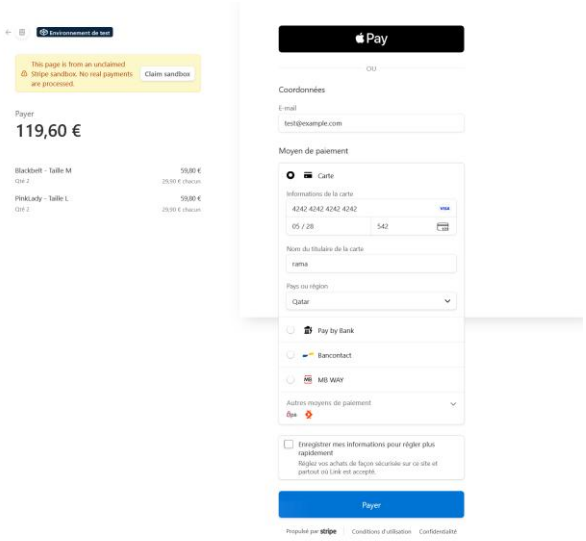
Après avoir cliqué sur « FINALISER MA COMMANDE », le navigateur a ouvert la page Stripe Checkout en environnement de test.

Les articles sélectionnés dans le panier apparaissaient avec leurs tailles, leurs quantités et le montant total de la commande.

Un paiement fictif a été réalisé à l'aide de la carte de test Stripe 4242 4242 4242 4242, avec une date d'expiration future et un code CVC fictif.

Le paiement a été accepté par Stripe, sans transaction bancaire réelle.

Figure 11.1 — Page Stripe Checkout affichant les articles et le montant de la commande.



11.4 Vérification et confirmation du paiement

Afin de vérifier le résultat du paiement, une méthode verifierPaiement() a été ajoutée dans src/Service/StripeService.php.

Cette méthode récupère la session de paiement auprès de Stripe et vérifie que son statut est paid.

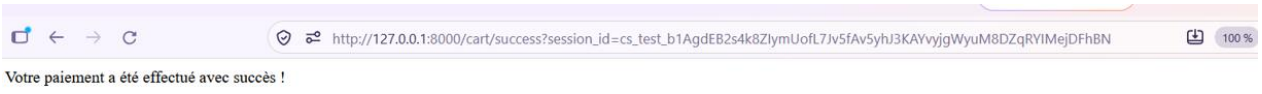
Le contrôleur CartController.php a également été complété avec la route /cart/success, associée à la méthode success().

Après un paiement accepté, Stripe redirige l'utilisateur vers cette route en transmettant l'identifiant de la session Checkout.

Symfony utilise cet identifiant pour interroger Stripe et confirmer le paiement.

Lorsque le statut du paiement est confirmé, le message « Votre paiement a été effectué avec succès ! » est affiché.

Figure 11.2 — Confirmation du paiement fictif Stripe sur le site STUBBORN.



11.5 Vérification du panier après paiement

Après la confirmation du paiement, le contrôleur supprime les articles enregistrés dans la session du panier.

Une vérification a été effectuée en retournant à l'adresse :

http://127.0.0.1:8000/cart

Le message « Votre panier est vide. » est alors apparu, confirmant que les articles avaient été retirés après le paiement.

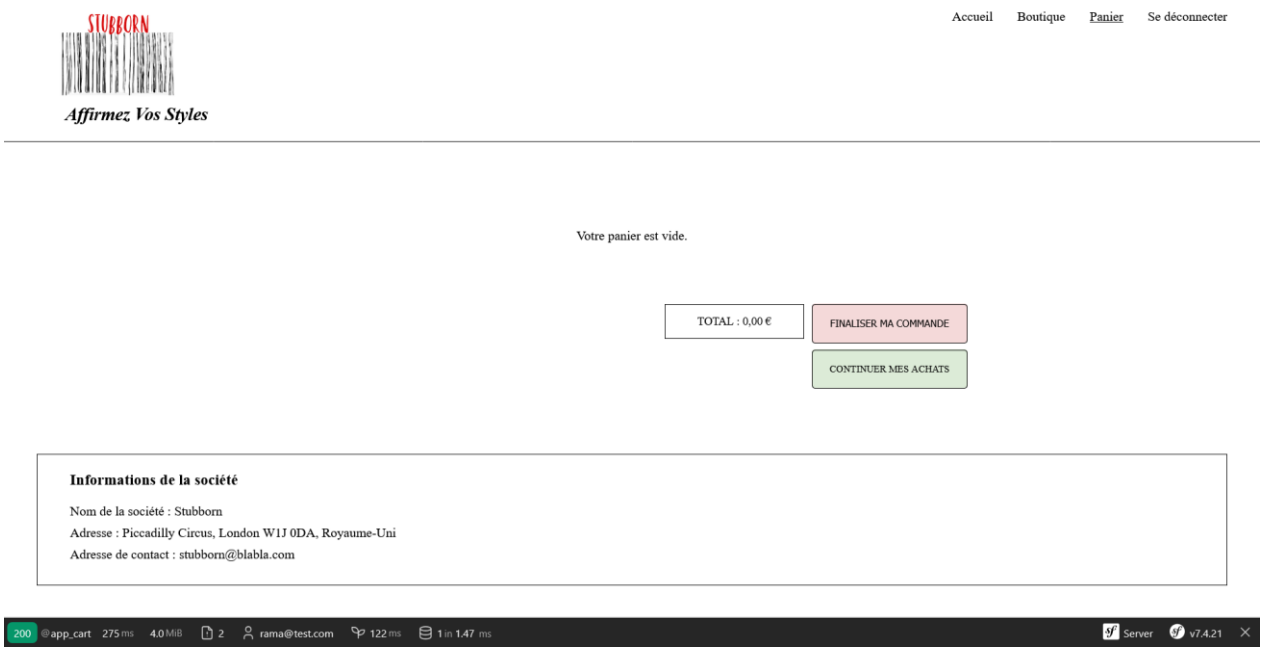


Figure 11.3 — Affichage du panier STUBBORN vide après la confirmation du paiement Stripe.

Ces tests ont permis de valider le parcours d'achat en environnement de développement, depuis la sélection des sweatshirts jusqu'à la confirmation du paiement fictif.

12 Réalisation du Back-Office – STUBBORN

12.1 Objectif du Back-Office

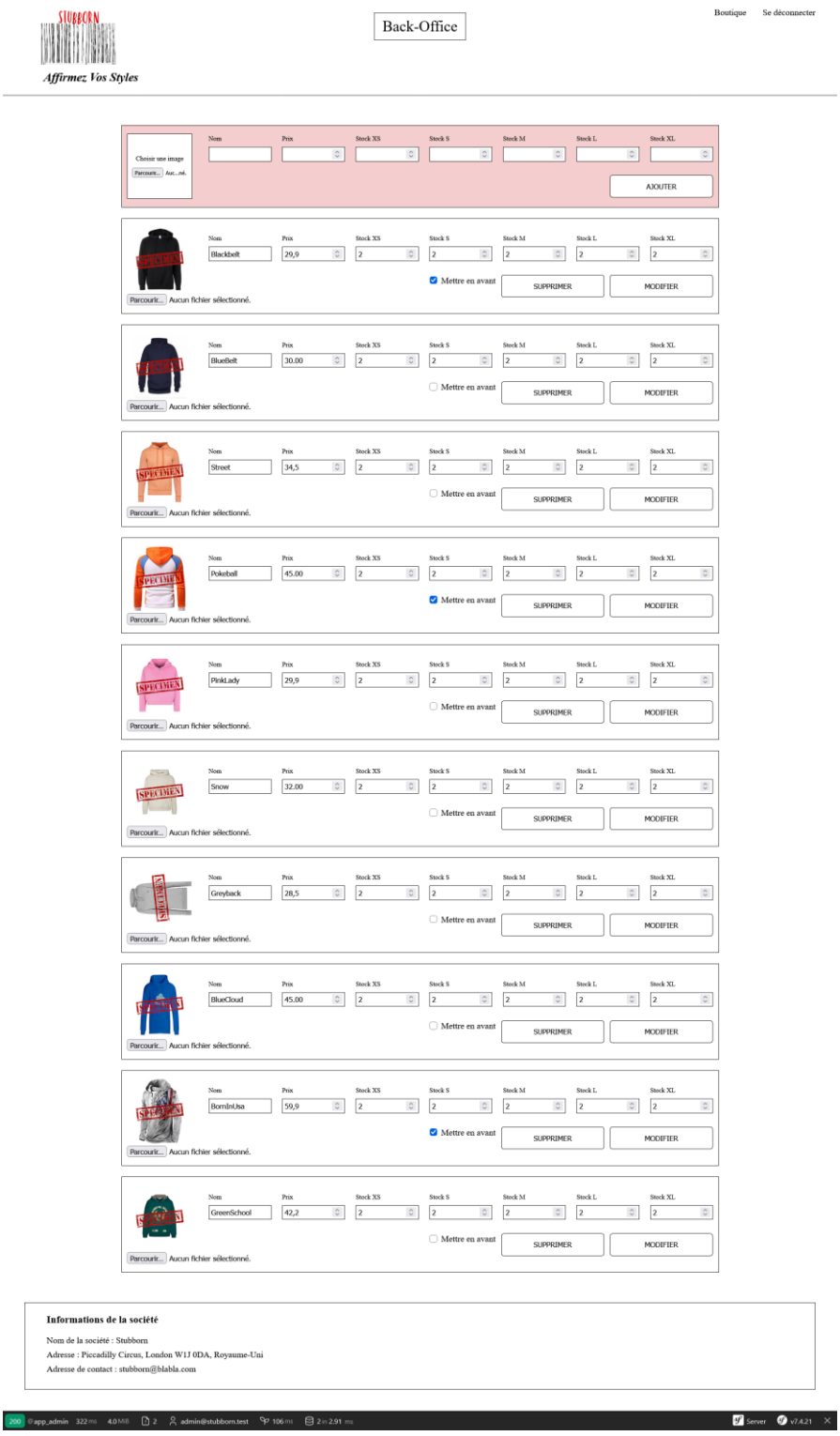
Le Back-Office est un espace d'administration permettant de gérer les sweatshirts proposés sur le site e-commerce STUBBORN.

Il est accessible uniquement aux utilisateurs disposant du rôle ROLE_ADMIN.

L'administrateur peut :

- Ajouter un nouveau sweatshirt avec son nom, son prix, son image et ses stocks.
- Modifier les informations d'un sweatshirt existant.
- Supprimer un sweatshirt.
- Sélectionner les sweatshirts à mettre en avant sur la page d'accueil.
- Consulter la Boutique pour vérifier les modifications effectuées.

Figure 12.1 — Vue générale du Back-Office STUBBORN



12.2 Sécurisation de l'accès administrateur

L'accès au Back-Office est protégé grâce au système de sécurité de Symfony.

Dans le fichier config/packages/security.yaml, une règle access_control limite l'accès aux routes commençant par /admin aux utilisateurs possédant le rôle ROLE_ADMIN.

Le compte administrateur est ainsi distingué des comptes clients classiques.

Après une connexion réussie, le fichier LoginSuccessHandler.php permet de rediriger automatiquement l'administrateur vers la page /admin, tandis qu'un client est redirigé vers la page d'accueil.

Figure 12.2 — Configuration de la protection du Back-Office

```yaml
    access_control:
        # BACK-OFFICE STUBBORN
        # Seuls les utilisateurs ayant le rôle ROLE_ADMIN
        # peuvent accéder à la page /admin et à ses sous-pages.
        - { path: ^/admin(?:/|$), roles: ROLE_ADMIN }

        # - { path: ^/profile, roles: ROLE_USER }
```

Figure 12.3 — Redirection après connexion

```php
// Déterminer la page à afficher après une connexion réussie
public function onAuthenticationSuccess(
    Request $request,
    TokenInterface $token
): RedirectResponse
{
    // Récupérer les rôles de l'utilisateur connecté
    $roles = $token->getRoleNames();

    // Rediriger l'administrateur vers le Back-Office
    if (in_array('ROLE_ADMIN', $roles, true)) {
        return new RedirectResponse(
            $this->urlGenerator->generate('app_admin')
        );
    }

    // Rediriger les clients vers la page d'accueil
    return new RedirectResponse(
        $this->urlGenerator->generate('app_home')
    );
}
```

12.3 Ajout d'un sweatshirt

Le formulaire AJOUTER permet à l'administrateur d'enregistrer un nouveau sweatshirt.

Les informations demandées sont le nom, le prix, une image et le stock disponible pour les tailles XS, S, M, L et XL.

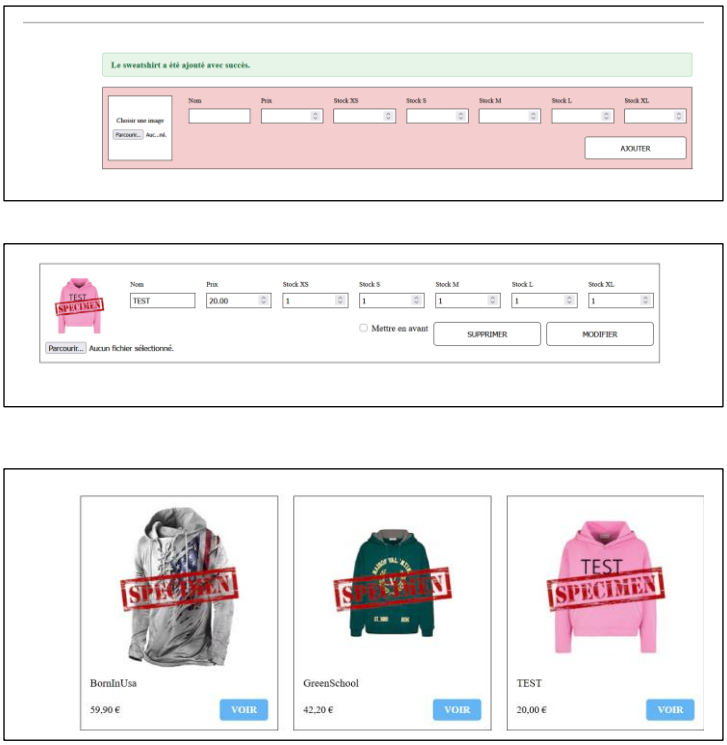
Les données sont vérifiées dans la méthode index() du fichier AdminController.php. Les images acceptées sont aux formats JPG, PNG ou WebP, avec une taille maximale de 5 Mo.

Après validation, Doctrine enregistre le produit dans la base de données. Un message confirme la réussite de l'opération.

Cette fonctionnalité a été testée avec un sweatshirt nommé TEST.

Figure 12.4 — Formulaire d'ajout d'un sweatshirt

Figure 12.5 — Confirmation de l'ajout



12.4 Modification d'un sweatshirt

Chaque sweatshirt possède un formulaire de modification contenant ses informations actuelles.

L'administrateur peut modifier le nom, le prix, les stocks et éventuellement remplacer l'image du produit.

Le bouton MODIFIER envoie les données à la route app_admin_product_edit, associée à la méthode edit() du contrôleur.

Les nouvelles informations sont enregistrées dans la base de données grâce à Doctrine.

Les modifications sont ensuite visibles sur le site Boutique.

Figure 12.6 — Formulaire de modification d'un sweatshirt



Figure 12.7 — modification vue dans page boutique client

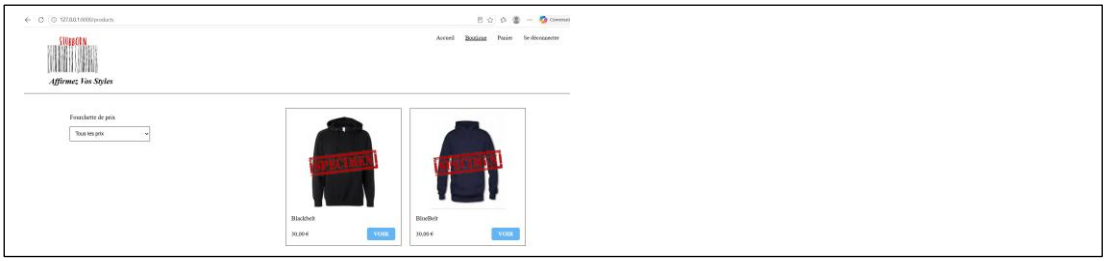
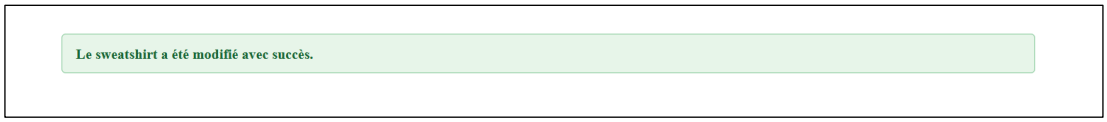


Figure 12.8 — Confirmation de la modification page Admin



Le Back-Office contient une case à cocher « Mettre en avant » pour chaque sweatshirt.

Cette case permet de définir les produits à afficher dans la section des produits mis en avant sur la page d'accueil.

La propriété isFeatured est enregistrée dans la base de données.

Le contrôleur de la page d'accueil récupère les produits sélectionnés et les transmet au template Twig pour leur affichage.

Figure 12.9 —Exemple de Sélection d'un sweatshirt à mettre en avant

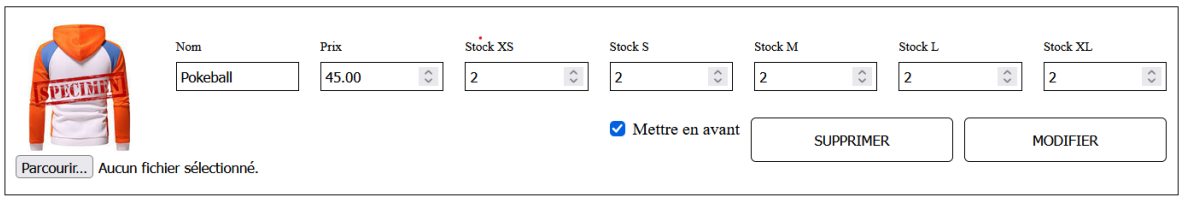
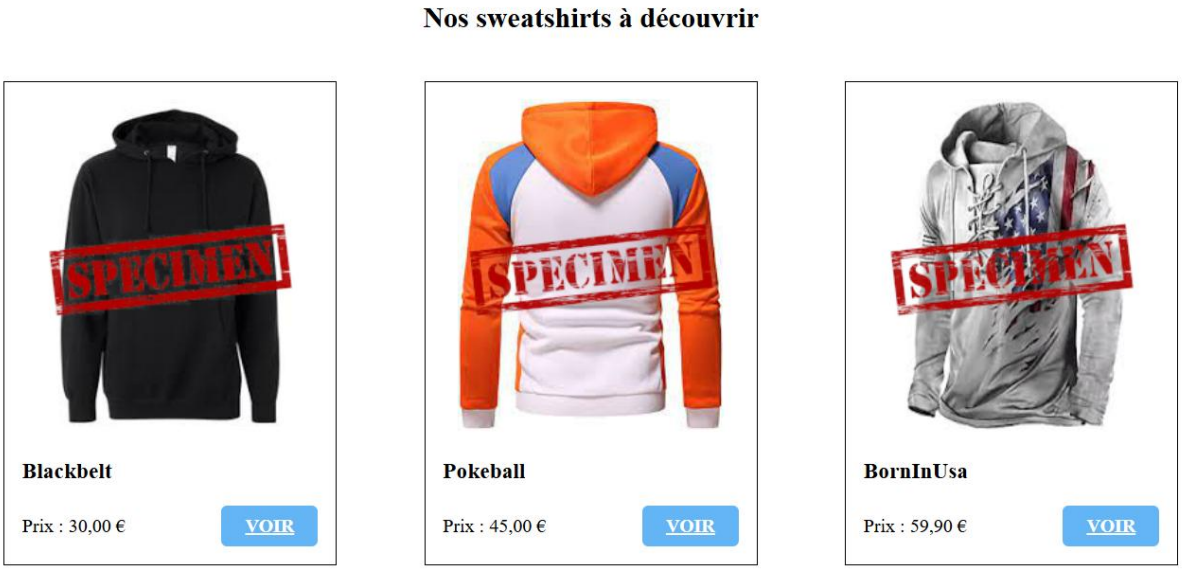


Figure 12.10 — Affichage des produits mis en avant sur l'accueil



12.6. Suppression d'un sweatshirt

Le bouton SUPPRIMER permet de retirer un sweatshirt de la base de données.

Pour sécuriser cette fonctionnalité, une route spécifique a été créée :

/admin/product/{id}/delete

Cette route accepte uniquement les requêtes POST et vérifie un jeton CSRF.

Avant la suppression, une fenêtre de confirmation demande à l'administrateur de valider son choix.

La suppression est réalisée dans la méthode delete() du fichier AdminController.php avec Doctrine.

Cette fonctionnalité a été testée avec le sweatshirt TEST. Après confirmation, le produit a disparu de la liste du Back-Office et un message de réussite a été affiché.

Figure 12.11 — Fenêtre de confirmation de suppression

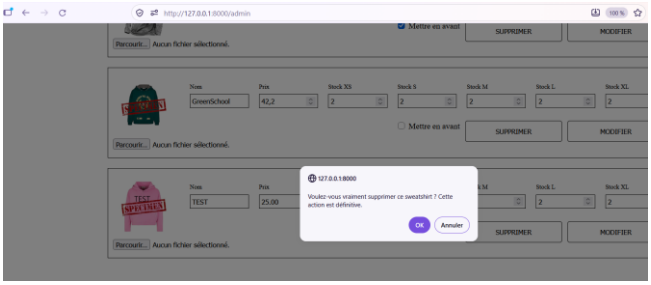


Figure 12.12 — Confirmation de la suppression

Le sweatshirt "TEST" a été supprimé avec succès.

12.7. Navigation entre le Back-Office et la Boutique

L'administrateur peut accéder à la Boutique depuis le menu du Back-Office afin de vérifier les produits affichés.

Un lien « Retour au Back-Office » a également été ajouté en bas de la page Boutique.

Ce lien utilise une condition Twig vérifiant le rôle ROLE_ADMIN. Il est donc visible uniquement pour l'administrateur et reste masqué pour les clients classiques.

Cette amélioration facilite la navigation sans modifier le menu principal prévu pour la Boutique.

Figure : Retour au back-office



← Retour au Back-Office

Informations de la société

Nom de la société : Stubborn
Adresse : Piccadilly Circus, London W1J 0DA, Royaume-Uni
Adresse de contact : stubborn@blabla.com

12.8. Vérifications techniques

Plusieurs vérifications ont été effectuées avec la console Symfony.

La commande php bin/console lint:twig templates/admin/index.html.twig a confirmé que le template Twig ne comportait pas d'erreur de syntaxe.

La commande php bin/console debug:router app_admin_product_delete a confirmé l'existence de la route de suppression et son utilisation de la méthode POST.

Les fonctionnalités d'ajout, de modification, de mise en avant et de suppression ont également fait l'objet de tests dans le navigateur.

12.9 Bilan

Le Back-Office de STUBBORN permet désormais à l'administrateur de gérer les sweatshirts depuis une interface dédiée.

Les principales fonctionnalités de gestion des produits ont été développées avec Symfony, Twig et Doctrine.

L'accès administrateur est protégé, les formulaires utilisent des jetons CSRF et les opérations réalisées sont accompagnées de messages de confirmation.

Les tests effectués ont permis de vérifier le fonctionnement des principales actions du Back-Office.

13 Réalisation des tests unitaires avec PHPUnit

13.1 Objectif des tests unitaires

Dans le cadre du projet e-commerce STUBBORN, des tests automatisés ont été réalisés avec PHPUnit afin de vérifier le fonctionnement du panier et du processus d'achat.

Ces tests permettent de contrôler automatiquement certains comportements de l'application, sans devoir effectuer manuellement les opérations dans le navigateur.

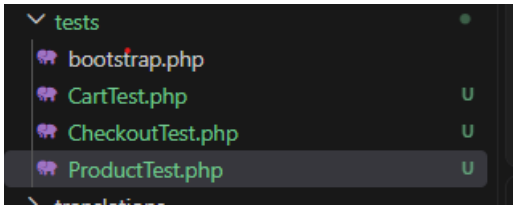
Les tests utilisent des données fictives et des dépendances simulées afin de ne pas modifier les produits enregistrés dans la base de données ni déclencher de véritables paiements Stripe.

L'environnement utilisé est le suivant :

- PHP 8.2.12
- Symfony 7.4
- PHPUnit 11.5.57
- Visual Studio Code
- Terminal PowerShell

Trois fichiers de tests ont été créés dans le dossier tests du projet.

Figure 13.1 — Organisation des fichiers de tests PHPUnit



13.2 Test du calcul du prix d'un produit

Le fichier ProductTest.php contient un premier test permettant de vérifier le calcul du montant total d'un produit en fonction de sa quantité.

Un sweatshirt fictif est créé avec un prix de 29 € et une quantité de deux exemplaires.

Le montant attendu est donc :

29 € × 2 = 58 €.

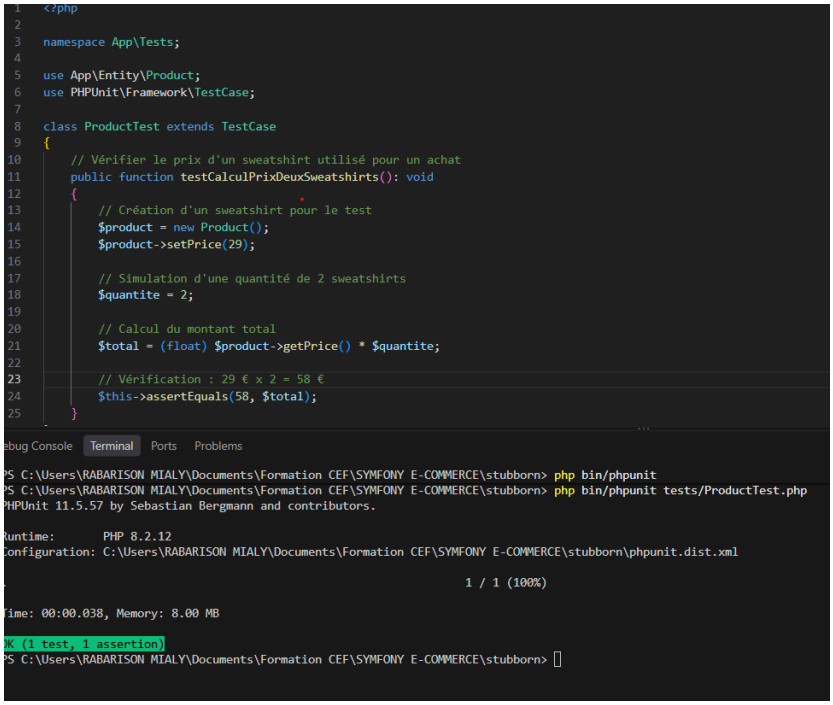
La méthode assertEquals() de PHPUnit permet de vérifier que le résultat obtenu correspond au montant attendu.

Le test a été exécuté avec la commande :

php bin/phpunit tests/ProductTest.php

Résultat obtenu : **OK (1 test, 1 assertion)**.

Figure 13.2 — Test du calcul du prix dans ProductTest.php



13.3 Test unitaire de l'ajout au panier

Le fichier CartTest.php permet de vérifier le fonctionnement de l'ajout d'un sweatshirt au panier.

Le scénario testé est le suivant :

- Un sweatshirt fictif possède un stock de cinq exemplaires en taille M.
- Un exemplaire est déjà présent dans le panier.
- Un deuxième exemplaire du même sweatshirt et de la même taille est ajouté.
- La quantité attendue dans le panier est de deux exemplaires.

Le test utilise la méthode add() du contrôleur CartController.php.

Le dépôt des produits, la validation CSRF et la génération des URL sont simulés afin de tester le comportement du contrôleur sans accéder à la base de données.

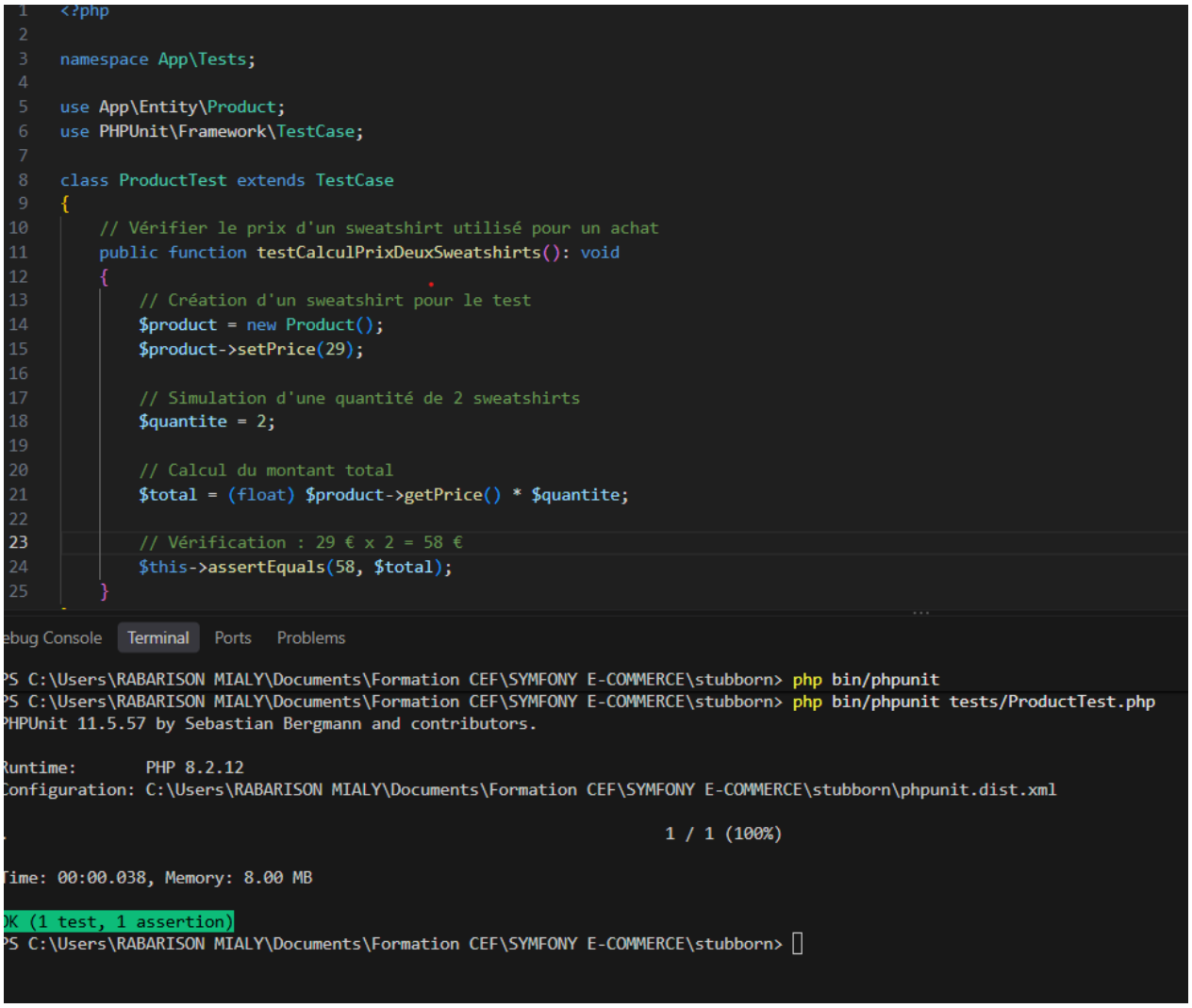
Les assertions vérifient notamment que la quantité du produit est correctement augmentée et que le client est redirigé vers la page du panier.

La commande utilisée est :

php bin/phpunit tests/CartTest.php

Résultat obtenu : **OK (1 test, 4 assertions)**.

Figure 13.3 — Test de l'ajout d'un sweatshirt au panier



13.4 Test d'achat avec un panier vide

Le fichier CheckoutTest.php contient un premier scénario vérifiant qu'un client ne peut pas déclencher un paiement lorsque son panier est vide.

Le test crée une session Symfony fictive contenant un panier vide.

Il appelle ensuite la méthode checkout() du contrôleur CartController.php.

Le service Stripe est simulé avec PHPUnit. Le test vérifie que la méthode de création d'une session de paiement n'est jamais appelée lorsque le panier est vide.

Il contrôle également que le client est redirigé vers la page du panier et que celui-ci reste vide.

Ce test permet de vérifier une règle de sécurité du processus d'achat : empêcher le lancement d'un paiement sans article valide.

Figure 13.4 — Test du refus de paiement avec un panier vide

```php
class CheckoutTest extends TestCase
{
    // Vérifier qu'un achat est refusé lorsque le panier est vide
    public function testAchatAvecPanierVide(): void
    {
        // Création d'une session de test avec un panier vide
        $session = new Session(new MockArraySessionStorage());
        $session->set('panier', []);
```

13.5 Test d'achat avec un produit valide

Un deuxième scénario a été ajouté dans CheckoutTest.php afin de vérifier la préparation d'un achat contenant des produits valides.

Le test utilise les données fictives suivantes :

- Produit : Sweatshirt TEST
- Prix unitaire : 29 €
- Taille : M
- Quantité : 2
- Stock disponible : 5

La méthode checkout() récupère les articles du panier et vérifie les informations du produit, sa taille, sa quantité et le stock disponible.

Le service Stripe est remplacé par une simulation PHPUnit.

Le test vérifie que le service reçoit le bon produit, la taille M et la quantité de deux exemplaires.

Il vérifie également que le contrôleur renvoie une redirection HTTP 303 vers une adresse de paiement Stripe simulée.

Enfin, il confirme que le panier reste présent avant la confirmation du paiement.

Aucune transaction réelle n'est effectuée pendant ce test.

Figure 13.5 — Test de préparation d'un achat valide

```php
// Vérifier la préparation d'un achat avec un sweatshirt valide
public function testAchatAvecProduitValide(): void
{
// Création d'un sweatshirt fictif pour le test
$product = new Product();
$product->setName('Sweatshirt TEST');
$product->setPrice(29);
$product->setStockM(5);

// Création d'une session contenant deux sweatshirts de taille M
$session = new Session(new MockArraySessionStorage());
$session->set('panier', [
    '1_M' => [
        'product_id' => 1,
        'taille' => 'M',
        'quantite' => 2,
    ],
]);

// Simulation de la demande de paiement
$request = Request::create('/cart/checkout', 'POST', [
    '_token' => 'token-test',
]);
$request->setSession($session);

// Simulation de la validation du jeton CSRF
$csrfManager = $this->createMock(CsrfTokenManagerInterface::class);
$csrfManager->method('isTokenValid')->willReturn(true);
```

Figure 13.6 — Vérification du paiement Stripe simulé

```php
// Simulation du service Stripe sans véritable paiement
$stripeService = $this->createMock(StripeService::class);

// Vérifier que Stripe reçoit les bons articles
$stripeService->expects($this->once())
    ->method('createCheckoutSession')
    ->with(
        $this->callback(
            function (array $articles) use ($product): bool {
                return count($articles) === 1
                    && $articles[0]['product'] === $product
                    && $articles[0]['taille'] === 'M'
                    && $articles[0]['quantite'] === 2;
            }
        ),
        $this->stringContains('/cart/success'),
        $this->stringContains('/cart')
    )
    ->willReturn(
        StripeCheckoutSession::constructFrom([
            'id' => 'cs_test_simulation',
            'url' => 'https://checkout.stripe.com/test-simulation',
        ])
    );
```

Figure 13.7 — Verification finale redirection Strip

```php
        // Vérification de la redirection vers Stripe
        $this->assertInstanceOf(RedirectResponse::class, $response);
        $this->assertSame(303, $response->getStatusCode());
        $this->assertSame(
            'https://checkout.stripe.com/test-simulation',
            $response->getTargetUrl()
        );

        // Le panier reste présent avant la confirmation du paiement
        $this->assertSame(2, $session->get('panier')['1_M']['quantite']);
    }
}
```

13.6 Exécution et résultats de l'ensemble des tests

Après avoir exécuté les tests individuellement, une vérification globale a été réalisée dans le terminal PowerShell avec la commande :

php bin/phpunit

Cette commande permet à PHPUnit de rechercher et d'exécuter l'ensemble des tests configurés dans le projet.

Le résultat obtenu est le suivant :

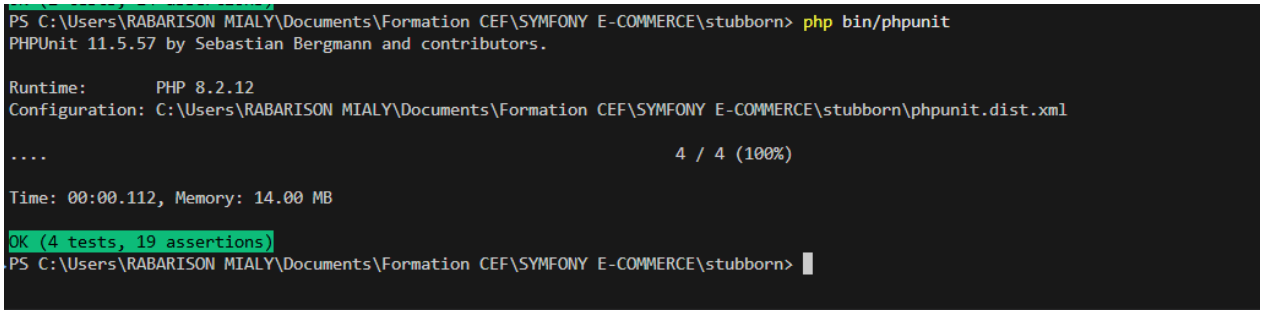
OK (4 tests, 19 assertions)

Les quatre tests ont été exécutés avec succès, sans erreur ni échec.

Les tests réalisés couvrent :

- Le calcul du prix d'un produit.
- L'ajout d'un deuxième exemplaire au panier.
- Le refus d'un achat avec un panier vide.
- La préparation d'un achat valide avec un service Stripe simulé.

Figure 13.8 — Résultat global de l'exécution des tests PHPUnit



13.7 Bilan des tests unitaires

La mise en place de PHPUnit a permis d'automatiser plusieurs vérifications essentielles du panier et du processus d'achat de STUBBORN. Les quatre tests ont été exécutés avec succès et ont validé 19 assertions. Ces tests complètent les essais manuels déjà réalisés dans le navigateur, notamment l'ajout de produits au panier et les achats-test avec Stripe.

Ils permettent de vérifier les scénarios développés et de faciliter la détection d'éventuelles régressions lors de futures modifications du code. Ils ne constituent toutefois pas une vérification exhaustive de tous les cas possibles ni un test réel de communication avec Stripe.